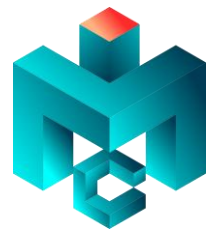




UNIVERSIDADE FEDERAL DE ITAJUBÁ
Instituto de Matemática e Computação



CMAC03 – Algoritmos em Grafos

6. Fluxo em Redes

Rafael Frinhani

frinhani@unifei.edu.br

1º Semestre de 2026

Apresentar os conceitos e algoritmos para o problema de fluxo máximo em redes e suas aplicações.

AGENDA

6. Fluxo em Redes

Introdução, Descrição, Definição, Aplicações, Problemas do Algoritmo Guloso para o Fluxo Máximo.

6.1. Algoritmo de Ford-Fulkerson

Contexto Histórico, Capacidade Residual da Rede, Princípio de funcionamento, Algoritmo, Exemplos, Casos Especiais.

6.2. Emparelhamento em grafos Bipartidos

Redes de Fluxo com múltiplas origens e sorvedouros, emparelhamento máximo em grafos bipartidos. Algoritmo Húngaro.





6. Fluxo em Redes

Introdução

Uma **rede** é a representação da estrutura de conexões entre componentes de um sistema. Em certas situações o **relacionamento entre os componentes caracteriza-se pelo movimento de materiais** (ex. veículos, água, eletricidade, dados etc).

fluxo = escoamento ou movimento contínuo de algo que segue em curso.

Os problemas de Fluxo em Redes tratam do movimento de materiais de um ponto de origem (oferta) até um ponto de destino (demanda) em um dado sistema.



6. Fluxo em Redes

Introdução

Uma rede é a representação da estrutura de conexões entre componentes de um sistema. Em certas situações o relacionamento entre os componentes caracteriza-se pelo movimento de materiais (ex. veículos, água, eletricidade, dados etc).

fluxo = escoamento ou movimento contínuo de algo que segue em curso.

Os problemas de Fluxo em Redes tratam do movimento de materiais de um ponto de origem (oferta) até um ponto de destino (demanda) em um dado sistema.

O grafo que representa a rede normalmente é direcionado e ponderado, mas o modelo pode considerar grafos simples e não-ponderados. Dois vértices especiais:

Fonte: ou origem (*source*) é o vértice sem arestas de entrada (apenas de saída) a partir do qual o fluxo tem início.

Destino: terminal ou sumidouro (*sink*) é o vértice sem arestas de saída (apenas de entrada) para onde o fluxo se destina.

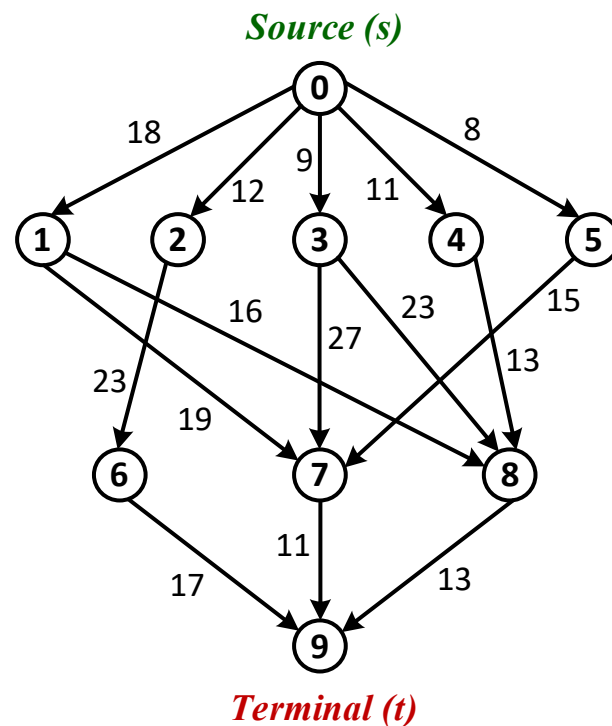


6. Fluxo em Redes

Descrição

Nos problemas de **Fluxo Máximo** o objetivo é enviar da fonte até o sumidouro o maior fluxo possível sem violar restrições de capacidade.

Um grafo neste tipo de problema é denominado rede de fluxo (*flow network*), tendo como características:





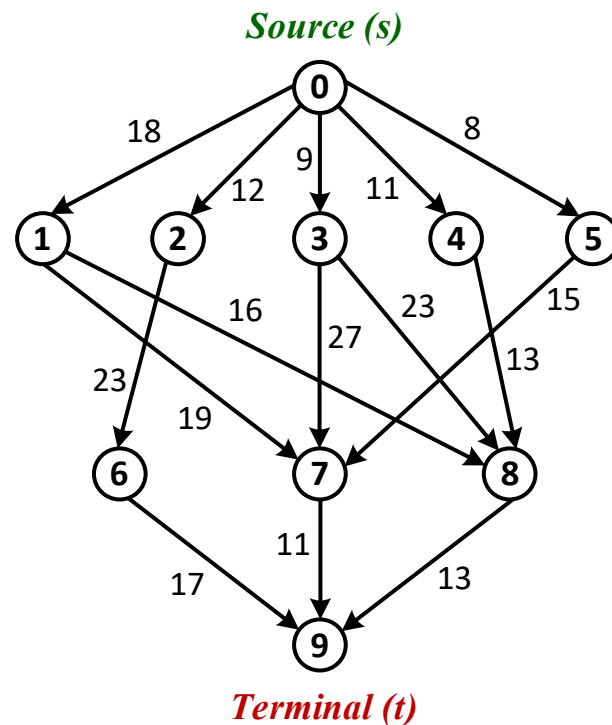
6. Fluxo em Redes

Descrição

Nos problemas de Fluxo Máximo o objetivo é enviar da fonte até o sumidouro o maior fluxo possível sem violar restrições de capacidade.

Um grafo neste tipo de problema é denominado rede de fluxo (*flow network*), tendo como características:

- Arcos representam canais por onde o material pode ser movimentado (ex. tubulação, estrada). Cada canal possui uma capacidade pré-estabelecida (peso do arco) dada como uma taxa máxima na qual o material pode fluir por ele.



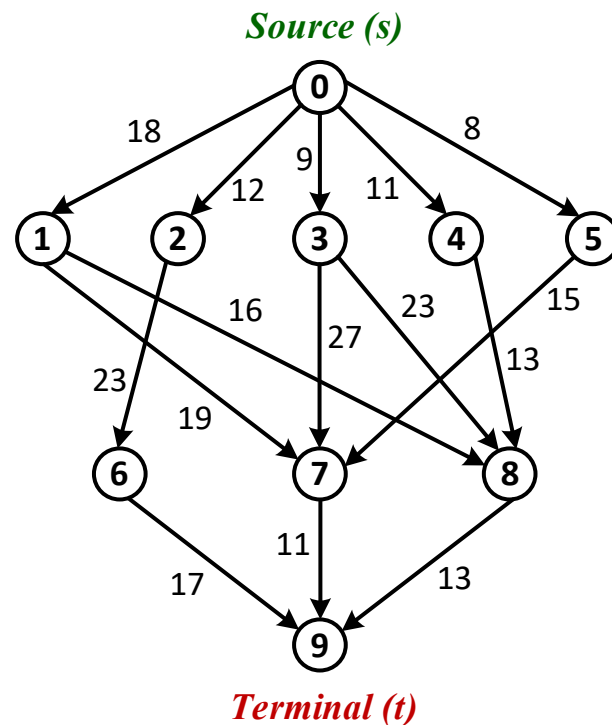


Descrição

Nos problemas de Fluxo Máximo o objetivo é enviar da fonte até o sumidouro o maior fluxo possível sem violar restrições de capacidade.

Um grafo neste tipo de problema é denominado rede de fluxo (*flow network*), tendo como **características**:

- Arcos representam canais por onde o material pode ser movimentado (ex. tubulação, estrada). Cada canal possui uma capacidade pré-estabelecida (peso do arco) dada como uma taxa máxima na qual o material pode fluir por ele.
- Vértices representam junções de canais (ex. válvula, cruzamento). Vértices intermediários ao fonte e sumidouro não possuem capacidade de armazenamento (**propriedade da conservação**, fluxo que entra = fluxo que sai).





6. Fluxo em Redes

Definição

Um fluxo em rede $G = (V, E)$ é um grafo orientado em que cada arco $(v, u) \in E$ tem **capacidade** positiva $c(i, j) \geq 0$. Caso não exista uma aresta entre v e u $c(i, j) = 0$.

Distingue-se dois vértices em um fluxo e rede: um **vértice origem** s considerado o **emissor** do fluxo, e um **vértice terminal** t considerado um **consumidor** do fluxo.



6. Fluxo em Redes

Definição

Um fluxo em rede $G = (V, E)$ é um grafo orientado em que cada arco $(v, u) \in E$ tem capacidade positiva $c(i, j) \geq 0$. Caso não exista uma aresta entre v e u $c(i, j) = 0$.

Distingue-se dois vértices em um fluxo e rede: um vértice origem s considerado o emissor do fluxo, e um vértice terminal t considerado um consumidor do fluxo.

Um fluxo em G é uma função de valor $f: V \times V \rightarrow R$ com as três **Propriedades**:

- **Restrição de Capacidade:** o fluxo de um vértice até outro não deve exceder a capacidade dada.

$$\forall v, u \in V, \text{ exige-se que } f(v, u) \leq c(v, u)$$

- **Anti-simetria oblíqua:** o fluxo de um vértice v até um vértice u é o valor negativo do fluxo no sentido inverso.

$$\forall v, u \in V, \text{ exige-se que } f(v, u) = -f(u, v)$$



6. Fluxo em Redes

Definição

Um fluxo em rede $G = (V, E)$ é um grafo orientado em que cada arco $(v, u) \in E$ tem capacidade positiva $c(i, j) \geq 0$. Caso não exista uma aresta entre v e u $c(i, j) = 0$.

Distingue-se dois vértices em um fluxo e rede: um vértice origem s considerado o emissor do fluxo, e um vértice terminal t considerado um consumidor do fluxo.

Um fluxo em G é uma função de valor $f: V \times V \rightarrow R$ com as três **Propriedades**:

- **Conservação de fluxo:** “fluxo que entra é igual o fluxo que sai”. O fluxo total para fora de um vértice que não seja origem ou terminal é 0.

$$\forall v, u \in V, \text{ exige-se que } \sum_{v \in V} f(v, u) = 0$$

O **valor de um fluxo** f é definido como

$$|f| = \sum_{v \in V} f(s, v)$$



6. Fluxo em Redes

Aplicações

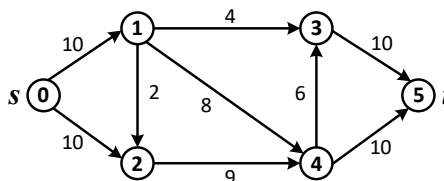
- Em projetos de redes hidráulicas (água, esgoto), de produtos químicos, de combustível (petróleo, gás).
- Dimensionamento de redes de transmissão de energia elétrica (baixa, média e alta tensão).
- Redes de computadores, de comunicação (ex. telefonia celular, rádio).
- Sistemas viários (estradas, rodovias), ferrovias, rotas aéreas, hidrovias.
- Problemas de manufatura em indústrias, planejamento de produção.
- Logística, cadeia de suprimentos, transporte de produtos.
- Estudos de dinâmica dos fluidos, escoamento, drenagem.



6. Fluxo em Redes

Algoritmo Guloso

Diferente de outros problemas modelados em grafos (ex. árvore geradora) os algoritmos gulosos não são adequados para fluxo máximo.



Estratégia gulosa:

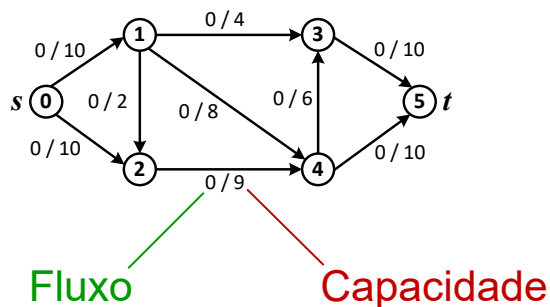
- Iniciar cada aresta com fluxo 0.
- Buscar um caminho $s \rightarrow t$ onde o fluxo das arestas é menor que a sua capacidade.
- Aumentar o fluxo do caminho.
- Repetir enquanto possível.



6. Fluxo em Redes

Algoritmo Guloso

Diferente de outros problemas modelados em grafos (ex. árvore geradora) os algoritmos gulosos não são adequados para fluxo máximo.



Estratégia gulosa:

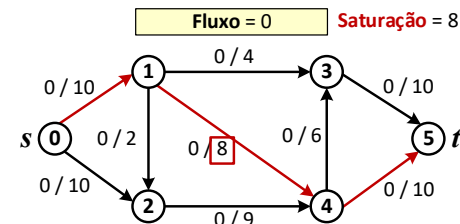
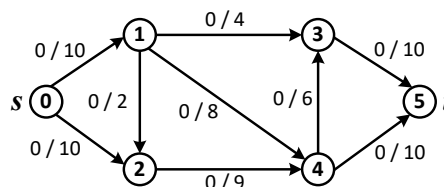
- Iniciar cada aresta com fluxo 0.
- Buscar um caminho $s \rightarrow t$ onde o fluxo das arestas é menor que a sua capacidade.
- Aumentar o fluxo do caminho.
- Repetir enquanto possível.



6. Fluxo em Redes

Algoritmo Guloso

Diferente de outros problemas modelados em grafos (ex. árvore geradora) os algoritmos gulosos não são adequados para fluxo máximo.



Estratégia gulosa:

- Iniciar cada aresta com fluxo 0.
- **Buscar um caminho $s \rightarrow t$ onde o fluxo das arestas é menor que a sua capacidade.**
- Aumentar o fluxo do caminho.
- Repetir enquanto possível.



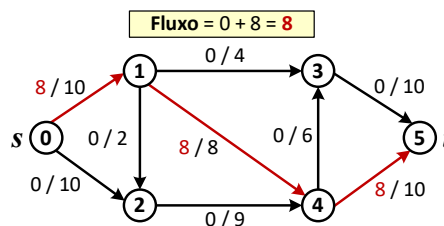
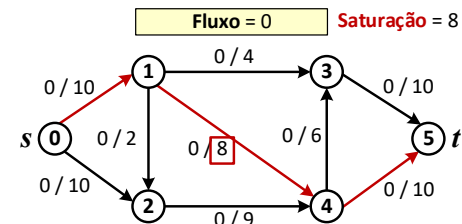
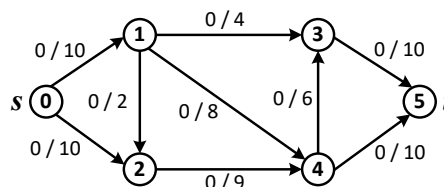
6. Fluxo em Redes

Algoritmo Guloso

Diferente de outros problemas modelados em grafos (ex. árvore geradora) os algoritmos gulosos não são adequados para fluxo máximo.

Estratégia gulosa:

- Iniciar cada aresta com fluxo 0.
- Buscar um caminho $s \rightarrow t$ onde o fluxo das arestas é menor que a sua capacidade.
- **Aumentar o fluxo do caminho.**
- Repetir enquanto possível.





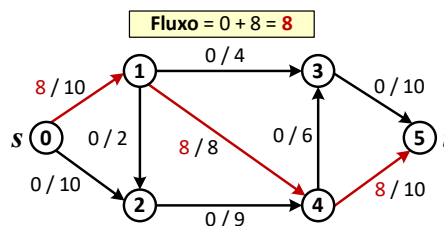
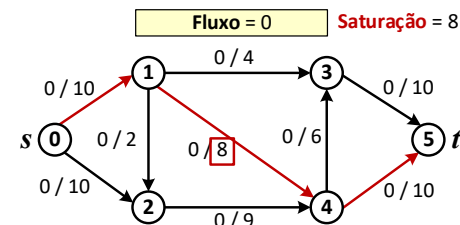
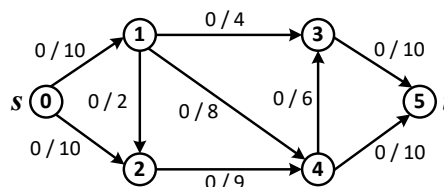
6. Fluxo em Redes

Algoritmo Guloso

Diferente de outros problemas modelados em grafos (ex. árvore geradora) os algoritmos gulosos não são adequados para fluxo máximo.

Estratégia gulosa:

- Iniciar cada aresta com fluxo 0.
- Buscar um caminho $s \rightarrow t$ onde o fluxo das arestas é menor que a sua capacidade.
- Aumentar o fluxo do caminho.
- Repetir enquanto possível.





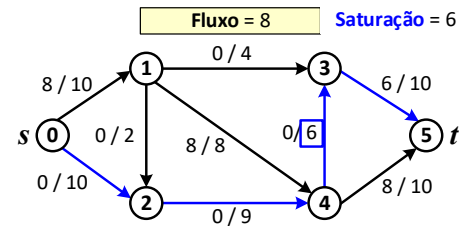
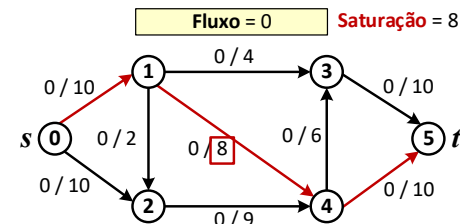
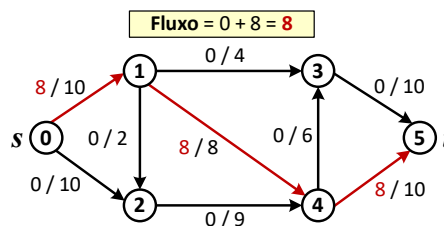
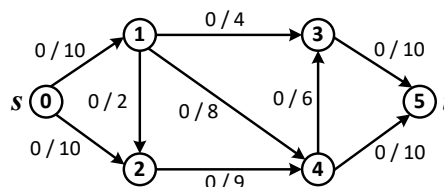
6. Fluxo em Redes

Algoritmo Guloso

Diferente de outros problemas modelados em grafos (ex. árvore geradora) os algoritmos gulosos não são adequados para fluxo máximo.

Estratégia gulosa:

- Iniciar cada aresta com fluxo 0.
- Buscar um caminho $s \rightarrow t$ onde o fluxo das arestas é menor que a sua capacidade.
- Aumentar o fluxo do caminho.
- Repetir enquanto possível.





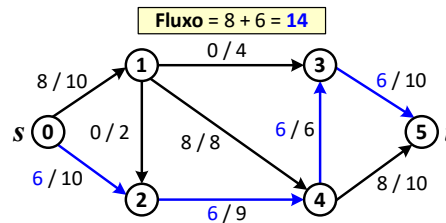
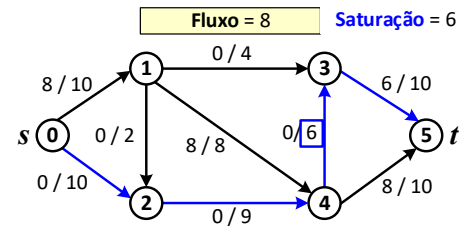
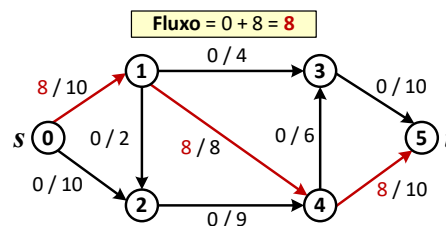
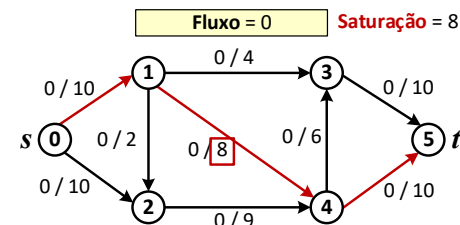
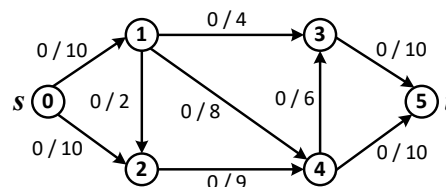
6. Fluxo em Redes

Algoritmo Guloso

Diferente de outros problemas modelados em grafos (ex. árvore geradora) os algoritmos gulosos não são adequados para fluxo máximo.

Estratégia gulosa:

- Iniciar cada aresta com fluxo 0.
- Buscar um caminho $s \rightarrow t$ onde o fluxo das arestas é menor que a sua capacidade.
- Aumentar o fluxo do caminho.
- Repetir enquanto possível.





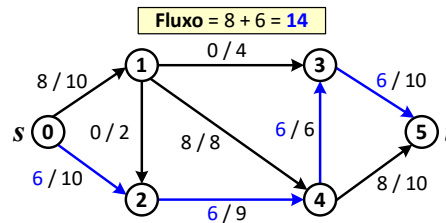
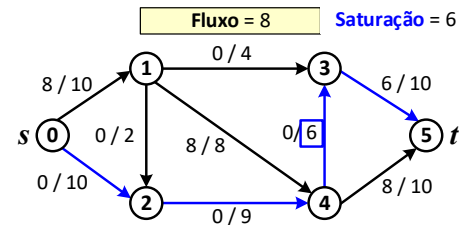
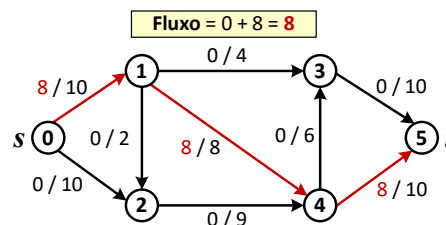
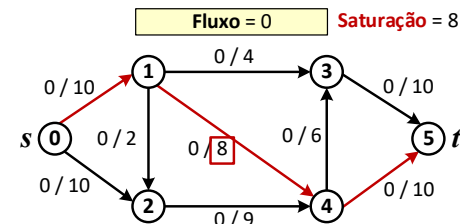
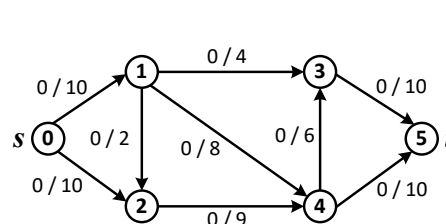
6. Fluxo em Redes

Algoritmo Guloso

Diferente de outros problemas modelados em grafos (ex. árvore geradora) os algoritmos gulosos não são adequados para fluxo máximo.

Estratégia gulosa:

- Iniciar cada aresta com fluxo 0.
- Buscar um caminho $s \rightarrow t$ onde o fluxo das arestas é menor que a sua capacidade.
- Aumentar o fluxo do caminho.
- Repetir enquanto possível.





6. Fluxo em Redes

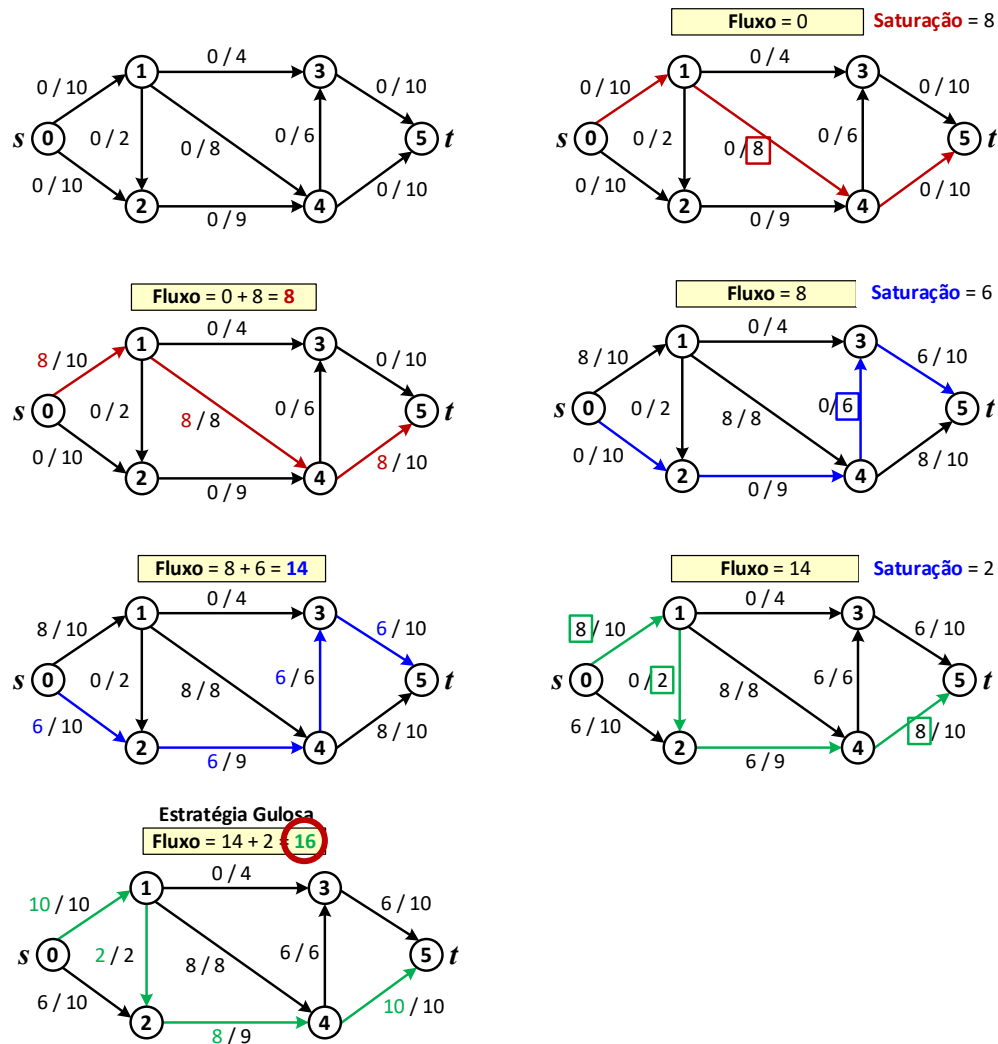
Algoritmo Guloso

Diferente de outros problemas modelados em grafos (ex. árvore geradora) os algoritmos gulosos não são adequados para fluxo máximo.

Estratégia gulosa:

- Iniciar cada aresta com fluxo 0.
- Buscar um caminho $s \rightarrow t$ onde o fluxo das arestas é menor que a sua capacidade.
- Aumentar o fluxo do caminho.
- Repetir enquanto possível.

O fluxo máximo da rede pelo algoritmo guloso é **16**.





6. Fluxo em Redes

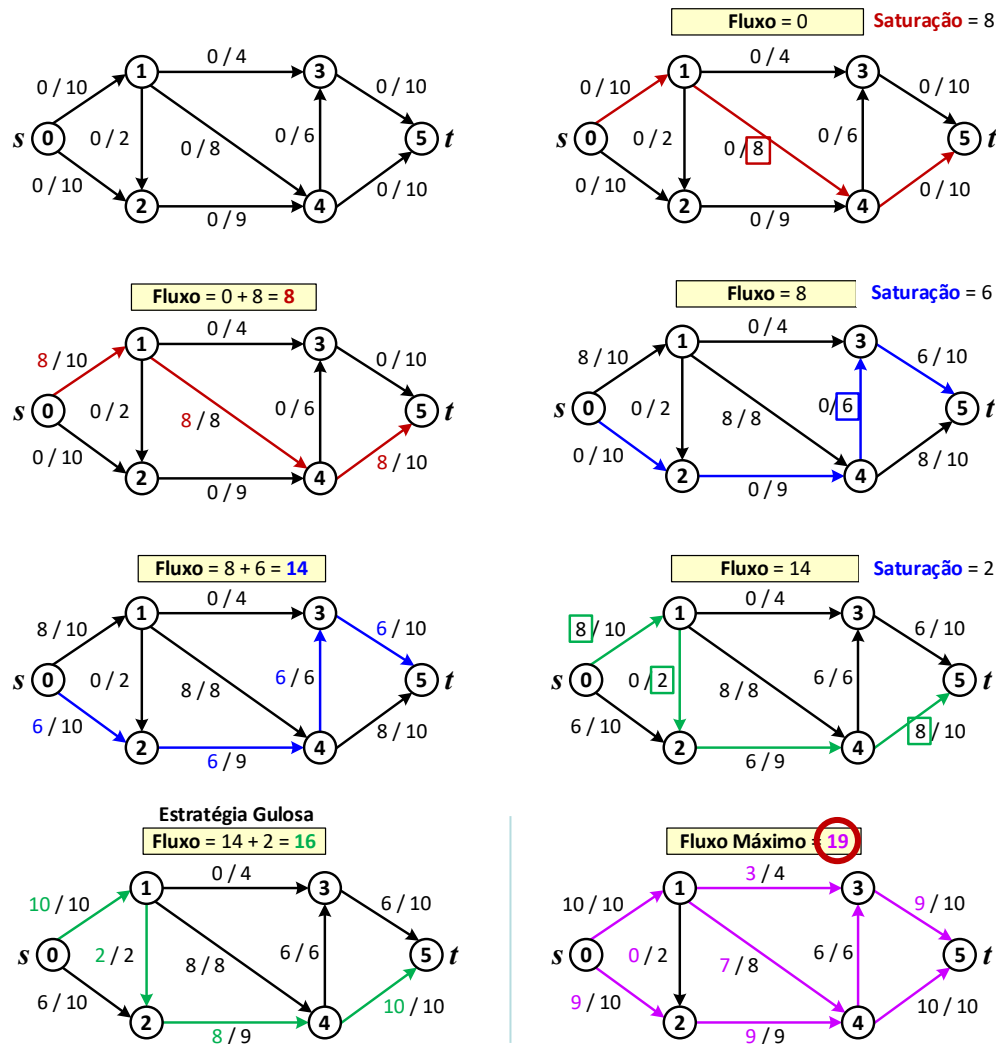
Algoritmo Guloso

Diferente de outros problemas modelados em grafos (ex. árvore geradora) os algoritmos gulosos não são adequados para fluxo máximo.

Estratégia gulosa:

- Iniciar cada aresta com fluxo 0.
- Buscar um caminho $s \rightarrow t$ onde o fluxo das arestas é menor que a sua capacidade.
- Aumentar o fluxo do caminho.
- Repetir enquanto possível.

Contudo, o **valor correto** do **fluxo máximo** da rede é **19** não é 16.



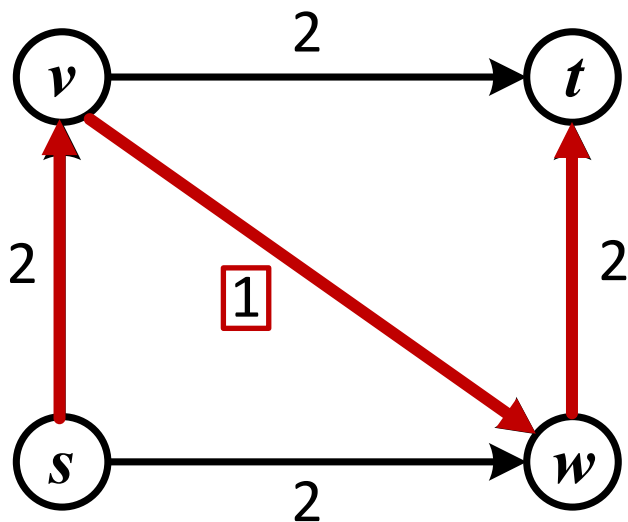


Algoritmo Guloso

O algoritmo guloso falha pois, uma vez que ele escolhe o caminho e aumenta o fluxo de seus arcos, ele **nunca diminui seu fluxo novamente para possíveis ajustes**.

Exemplo:

- O algoritmo guloso pode optar primeiro pelo caminho $s - v - w - t$, mas esta é uma decisão ruim pois o canal (v, w) é um gargalo da rede.
- Para uma melhor solução, é necessário algum mecanismo para “desfazer” decisões ruins.



Fluxo Máximo = 3



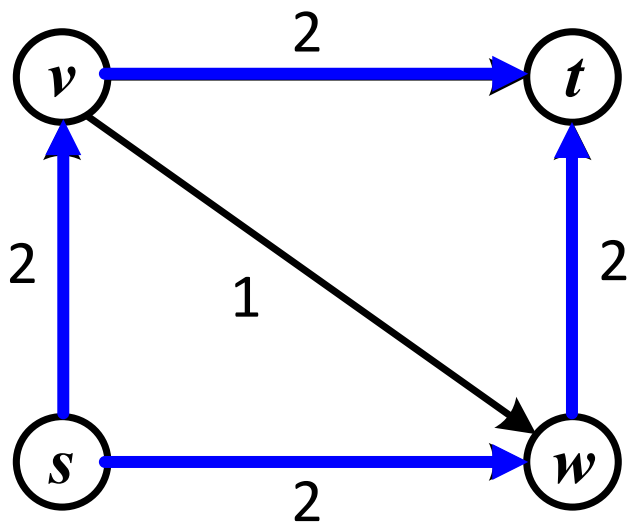
6. Fluxo em Redes

Algoritmo Guloso

O algoritmo guloso falha pois, uma vez que ele escolhe o caminho e aumenta o fluxo de seus arcos, ele **nunca diminui seu fluxo novamente para possíveis ajustes**.

Exemplo:

- O algoritmo guloso pode optar primeiro pelo caminho $s - v - w - t$, mas esta é uma decisão ruim pois o canal (v, w) é um gargalo da rede.
- Para uma melhor solução, é necessário algum mecanismo para “desfazer” decisões ruins.
- Os caminhos $s - v - t$ e $s - w - t$ correspondem ao fluxo máximo.
- No fluxo máximo de $s - t$ da rede ao lado, o fluxo $f(v, w) = 0$, pois ele não é utilizado.



Fluxo Máximo = 4



6. Fluxo em Redes

Algoritmos

A história dos algoritmos de fluxo está relacionada à análise da rede ferroviária da União Soviética entre as décadas de 1930 e 1950. [Schrijver, Alexander, "On the history of the transportation and maximum flow problems". *Mathematical Programming*. 91: 437-445, 2002.](#)

Ford-Fulkerson: Definiu um algoritmo para solução do fluxo máximo. Não especifica o procedimento de busca.



6. Fluxo em Redes

Algoritmos

A história dos algoritmos de fluxo está relacionada à análise da rede ferroviária da União Soviética entre as décadas de 1930 e 1950. Schrijver, Alexander, "On the history of the transportation and maximum flow problems". *Mathematical Programming*. **91**: 437-445, 2002.

Ford-Fulkerson: Definiu um algoritmo para solução do fluxo máximo. Não especifica o procedimento de busca.

Edmonds-Karp: Baseado no algoritmo de Ford-Fulkerson, **emprega BFS como método para encontrar os caminhos de aumento de fluxo**. Possui complexidade total igual a $O(VE^2)$.

Dinic: Também baseado no Ford-Fulkerson, usa uma **combinação de DFS e BFS para encontrar os caminhos de aumento**. Complexidade total é $O(V^2E)$.

As complexidades de tempo dos algoritmos acima são pessimistas. Na prática, os algoritmos tendem a executar mais rapidamente, tornando difícil compará-los apenas por sua complexidade assintótica.



Algoritmo de Ford-Fulkerson

Proposto originalmente por Lester Randolph Jr e Delbert Ray Fulkerson em 1962 para solução do problema de determinação do fluxo máximo.

Princípio de Funcionamento

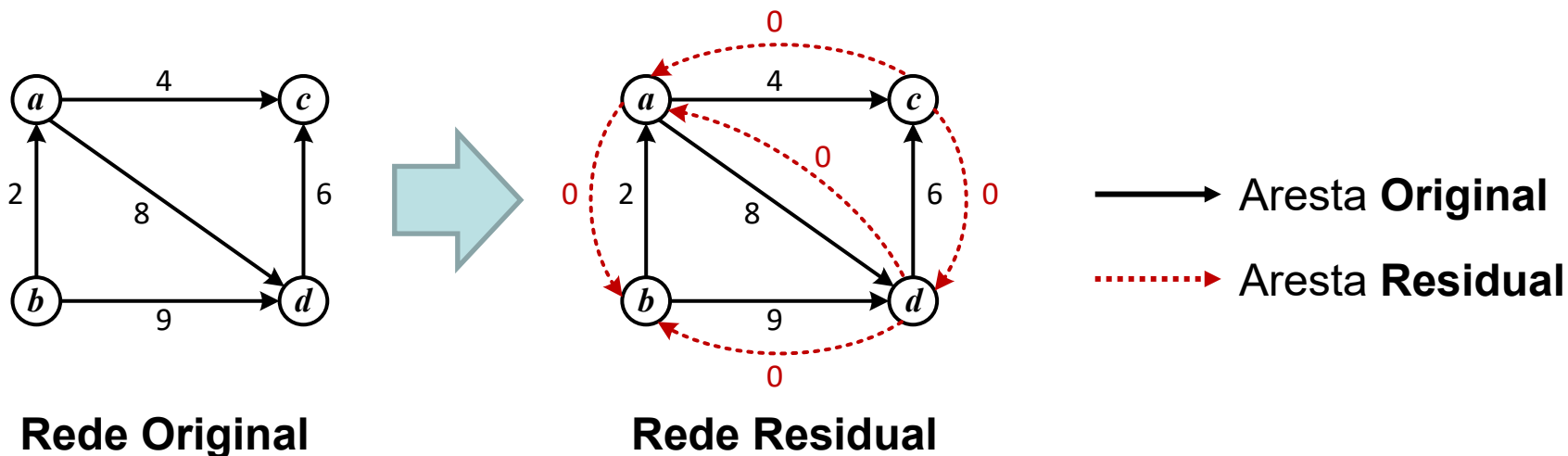
- O método considera inicialmente um fluxo vazio na rede, ou seja, o fluxo de cada um dos canais é zero.
- Através de um procedimento de busca (ex. BFS, DFS, Dijkstra etc), a cada passo o algoritmo encontra um caminho da origem até o destino, de modo a aumentar o fluxo.
- Quando o algoritmo não é mais capaz de aumentar o fluxo, significa que o fluxo máximo da rede foi atingido.



6.1. Ford-Fulkerson

Capacidade Residual da Rede

- O método a seguir usa uma representação especial de grafo, na qual cada aresta original da rede possui uma aresta correspondente em sentido contrário, denominada **aresta residual**.

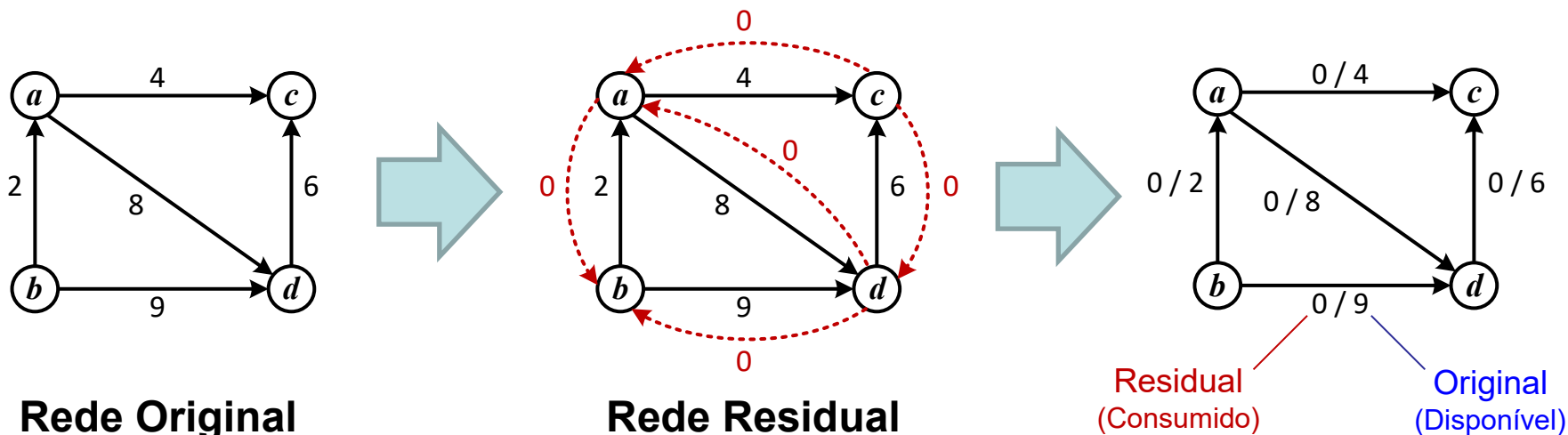




6.1. Ford-Fulkerson

Capacidade Residual da Rede

- O método a seguir usa uma representação especial de grafo, na qual cada aresta original da rede possui uma aresta correspondente em sentido contrário, denominada aresta residual.
- O **peso** de cada aresta (original ou residual) **indica o quanto de fluxo ainda é possível trafegar** por ela (capacidade do canal).
- No início do algoritmo, o peso de cada aresta original é igual a sua capacidade, e o peso de cada aresta residual é igual a zero. A **soma das arestas original e residual** é sempre **igual a capacidade total do canal** que elas representam.





6.1. Ford-Fulkerson

Funcionamento

- O algoritmo executa várias iterações sobre o grafo.
- A **cada iteração**, ele **escolhe um caminho da origem até o destino** de forma que o peso das arestas do caminho sejam positivos (maior que zero).
 - Se existir mais de um caminho, qualquer um deles pode ser analisado naquela iteração.
 - Qualquer aresta (original ou residual) pode fazer parte do caminho.



6.1. Ford-Fulkerson

Funcionamento

- O algoritmo executa várias iterações sobre o grafo.
- A cada iteração, ele escolhe um caminho da origem até o destino de forma que o peso das arestas do caminho sejam positivos (maior que zero).
 - Se existir mais de um caminho, qualquer um deles pode ser analisado naquela iteração.
 - Qualquer aresta (original ou residual) pode fazer parte do caminho.
- **Após escolher o caminho:**
 - O **fluxo do canal** é **X** unidades, sendo que X é o menor peso de aresta do caminho (canal que caracteriza um **gargalo**).
 - O **peso das arestas originais** do caminho **diminui** em X unidades (representa o valor ainda disponível no canal).
 - O **peso das arestas residuais** do caminho **aumenta** em X unidades (valor consumido do canal).



6.1. Ford-Fulkerson

Funcionamento

- A ideia é que, o **aumento do fluxo em um caminho diminui** a quantidade de **fluxo que pode percorrer** as respectivas **arestas** no futuro.
- Ao mesmo tempo, é possível “cancelar” **fluxos** em passos subsequentes **usando as arestas residuais (reversas)**, se o algoritmo detectar que esta ação pode beneficiar a injeção de fluxo em outro caminho.

Interpretação: não se “caminhou” por uma aresta em seu sentido contrário, apenas “deixou-se” de injetar uma certa quantia de fluxo por ela. A capacidade ainda disponível no canal é representada pela aresta original.



Funcionamento

- A ideia é que, o aumento do fluxo em um caminho diminui a quantidade de fluxo que pode percorrer as respectivas arestas no futuro.
- Ao mesmo tempo, é possível “cancelar” fluxos em passos subsequentes usando as arestas residuais (reversas), se o algoritmo detectar que esta ação pode beneficiar a injeção de fluxo em outro caminho.

Interpretação: não se “caminhou” por uma aresta em seu sentido contrário, apenas “deixou-se” de injetar uma certa quantia de fluxo por ela. A capacidade ainda disponível no canal é representada pela aresta original.

- O algoritmo aumenta o fluxo enquanto existirem caminhos do vértice de origem ao vértice de destino que passem apenas por arestas com pesos positivos maior que zero.
- Não havendo mais caminhos o algoritmo encerra e o fluxo máximo foi encontrado.



6.1. Ford-Fulkerson

Algoritmo

fordFulkerson(G, s, t)

```
1  $G_r \leftarrow G$ ;  
2  $fluxoMax \leftarrow 0$ ;  
3 while  $\exists$  caminho  $p$  de  $s-t$  em  $G_r$  do  
4    $c_r(p) \leftarrow \min\{c_r(v, u) : (v, u) \in p\}$ ;  
5   for cada arco  $(v, u) \in p$  do  
6      $G_r[v][u] \leftarrow G_r[v][u] - c_r(p)$ ;  
7      $G_r[u][v] \leftarrow G_r[v][u] + c_r(p)$ ;  
8   end  
9    $fluxoMax \leftarrow fluxoMax + c_r(p)$ ;  
10 end  
11 return  $fluxoMax$ 
```

ESTRUTURAS

- G, G_r : Rede de Fluxo G e respectiva Rede Residual G_r .
- $c_r(p)$: capacidade residual de um caminho p
- $G_r[v][u]$: aresta (v, u) da Rede Residual G_r

Definir a rede residual G_r como uma cópia da rede de fluxo G e inicializar a variável $fluxoMax$, que armazenará o fluxo máximo da rede residual (linhas 1 e 2).



6.1. Ford-Fulkerson

Algoritmo

fordFulkerson(G, s, t)

```
1  $G_r \leftarrow G$ ;  
2  $fluxoMax \leftarrow 0$ ;  
3 while  $\exists$  caminho  $p$  de  $s-t$  em  $G_r$  do  
4    $c_r(p) \leftarrow \min\{c_r(v, u) : (v, u) \in p\}$ ;  
5   for cada arco  $(v, u) \in p$  do  
6      $G_r[v][u] \leftarrow G_r[v][u] - c_r(p)$ ;  
7      $G_r[u][v] \leftarrow G_r[v][u] + c_r(p)$ ;  
8   end  
9    $fluxoMax \leftarrow fluxoMax + c_r(p)$ ;  
10 end  
11 return  $fluxoMax$ 
```

ESTRUTURAS

- G, G_r : Rede de Fluxo G e respectiva Rede Residual G_r
- $c_r(p)$: capacidade residual de um caminho p
- $G_r[v][u]$: aresta (v, u) da Rede Residual G_r

Definir a rede residual G_r como uma cópia da rede de fluxo G e inicializar a variável $fluxoMax$, que armazenará o fluxo máximo da rede residual (linhas 1 e 2).

Enquanto existirem caminhos entre $(s - t)$ na rede residual G_r (linhas 3 a 10):

Encontrar o valor mínimo (gargalo) da capacidade residual do caminho p (linha 4).

Para cada aresta do caminho p (linha 5) atualizar a capacidade das arestas originais (linha 6) e das arestas residuais (linha 7) de G_r .

Atualizar o valor do fluxo máximo da rede (linha 9).

Retornar o fluxo máximo da rede G_r .



6.1. Ford-Fulkerson

Algoritmo – Exemplo 1

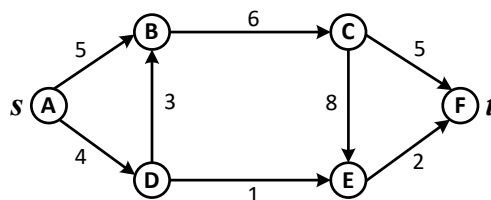
fordFulkerson(G, s, t)

```
1  $G_r \leftarrow G$ ;  
2  $fluxoMax \leftarrow 0$ ;  
3 while  $\exists$  caminho  $p$  de  $s$ – $t$  em  $G_r$  do  
4    $c_r(p) \leftarrow \min\{c_r(v, u) : (v, u) \in p\}$ ;  
5   for cada arco  $(v, u) \in p$  do  
6      $G_r[v][u] \leftarrow G_r[v][u] - c_r(p)$ ;  
7      $G_r[u][v] \leftarrow G_r[v][u] + c_r(p)$ ;  
8   end  
9    $fluxoMax \leftarrow fluxoMax + c_r(p)$ ;  
10 end  
11 return  $fluxoMax$ 
```

Caminho: –

Fluxo = 0

G



Uma rede de fluxo G com vértice A como origem e F como destino.



6.1. Ford-Fulkerson

Algoritmo – Exemplo 1

fordFulkerson(G, s, t)

```

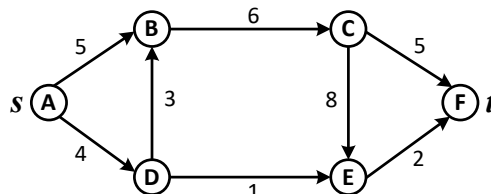
1  $G_r \leftarrow G$ ;
2  $fluxoMax \leftarrow 0$ ;
3 while  $\exists$  caminho  $p$  de  $s-t$  em  $G_r$  do
4    $c_r(p) \leftarrow \min\{c_r(v, u) : (v, u) \in p\}$ ;
5   for cada arco  $(v, u) \in p$  do
6      $G_r[v][u] \leftarrow G_r[v][u] - c_r(p)$ ;
7      $G_r[u][v] \leftarrow G_r[u][v] + c_r(p)$ ;
8   end
9    $fluxoMax \leftarrow fluxoMax + c_r(p)$ ;
10 end
11 return  $fluxoMax$ 

```

Caminho: –

Fluxo = 0

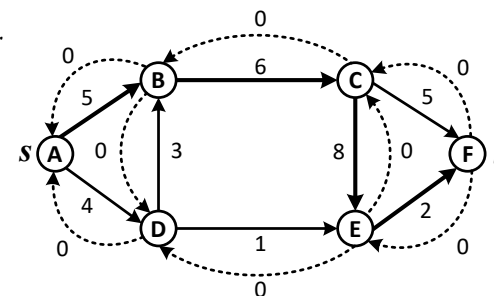
G



Caminho:

Fluxo = 0

G_r



Uma rede de fluxo G com vértice A como origem e F como destino.

É criada uma rede residual G_r a partir da rede de fluxo G . O fluxo máximo é inicializado com valor 0 (Fluxo = 0).



6.1. Ford-Fulkerson

Algoritmo – Exemplo 1

fordFulkerson(G, s, t)

```

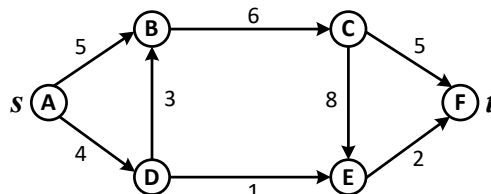
1  $G_r \leftarrow G$ ;
2  $fluxoMax \leftarrow 0$ ;
3 while  $\exists$  caminho  $p$  de  $s$ – $t$  em  $G_r$  do
4    $c_r(p) \leftarrow \min\{c_r(v, u) : (v, u) \in p\}$ ;
5   for cada arco  $(v, u) \in p$  do
6      $G_r[v][u] \leftarrow G_r[v][u] - c_r(p)$ ;
7      $G_r[u][v] \leftarrow G_r[u][v] + c_r(p)$ ;
8   end
9    $fluxoMax \leftarrow fluxoMax + c_r(p)$ ;
10 end
11 return  $fluxoMax$ 

```

Caminho: –

Fluxo = 0

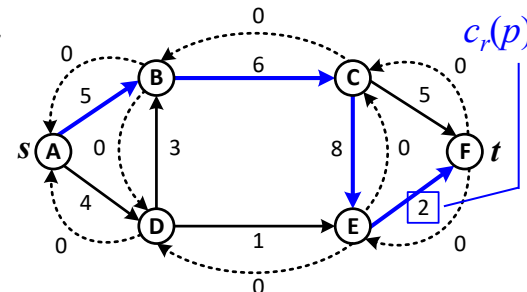
G



G_r

Caminho: A – B – C – E – F

Fluxo = 0



Uma rede de fluxo G com vértice A como origem e F como destino.

É criada uma rede residual G_r a partir da rede de fluxo G . O fluxo máximo é inicializado com valor 0 (Fluxo = 0).

A capacidade residual do caminho destacado na rede é $c_r(p) = 2$.



6.1. Ford-Fulkerson

Algoritmo – Exemplo 1

fordFulkerson(G, s, t)

```

1  $G_r \leftarrow G$ ;
2  $fluxoMax \leftarrow 0$ ;
3 while  $\exists$  caminho  $p$  de  $s-t$  em  $G_r$  do
4    $c_r(p) \leftarrow \min\{c_r(v, u) : (v, u) \in p\}$ ;
5   for cada arco  $(v, u) \in p$  do
6      $G_r[v][u] \leftarrow G_r[v][u] - c_r(p)$ ;
7      $G_r[u][v] \leftarrow G_r[u][v] + c_r(p)$ ;
8   end
9    $fluxoMax \leftarrow fluxoMax + c_r(p)$ ;
10 end
11 return  $fluxoMax$ 

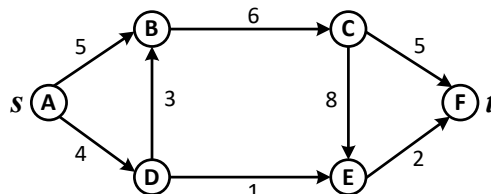
```

Atualiza cada aresta original e residual do caminho p conforme a capacidade residual $c_r(p) = 2$. Também atualiza o valor do fluxo.

Caminho: –

Fluxo = 0

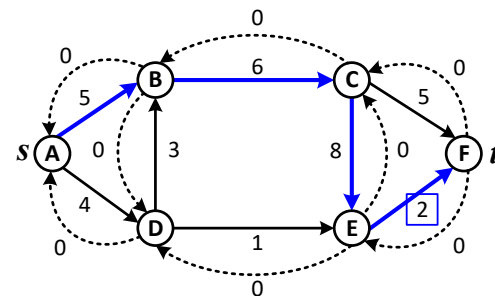
G



G_r

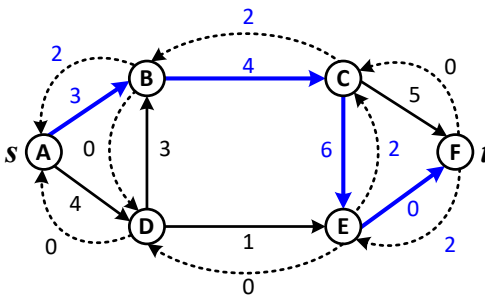
Caminho: $A-B-C-E-F$

Fluxo = 0



Caminho: $A-B-C-E-F$

Fluxo = $0 + 2 = 2$





6.1. Ford-Fulkerson

Algoritmo – Exemplo 1

fordFulkerson(G, s, t)

```

1  $G_r \leftarrow G$ ;
2  $fluxoMax \leftarrow 0$ ;
3 while  $\exists$  caminho  $p$  de  $s-t$  em  $G_r$  do
4    $c_r(p) \leftarrow \min\{c_r(v, u) : (v, u) \in p\}$ ;
5   for cada arco  $(v, u) \in p$  do
6      $G_r[v][u] \leftarrow G_r[v][u] - c_r(p)$ ;
7      $G_r[u][v] \leftarrow G_r[v][u] + c_r(p)$ ;
8   end
9    $fluxoMax \leftarrow fluxoMax + c_r(p)$ ;
10 end
11 return  $fluxoMax$ 

```

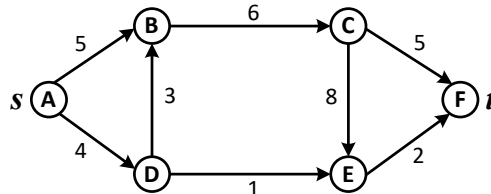
Atualiza cada aresta original e residual do caminho p conforme a capacidade residual $c_r(p) = 2$. Também atualiza o valor do fluxo.

Reinicia a partir de outro caminho, cujo fluxo está limitado ao canal mais restrito $c_r(p) = 3$.

Caminho: –

Fluxo = 0

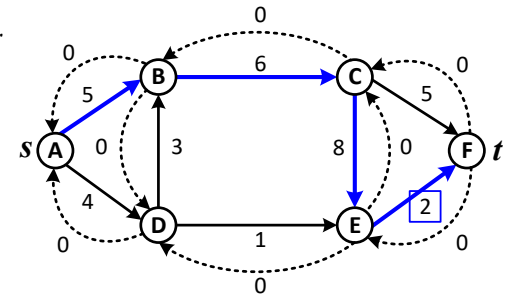
G



G_r

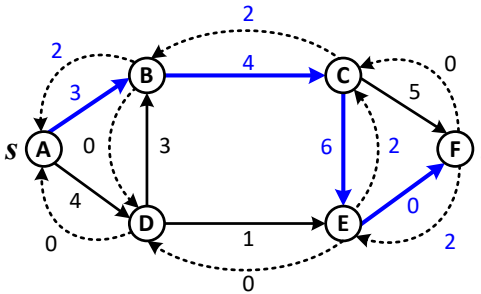
Caminho: A – B – C – E – F

Fluxo = 0



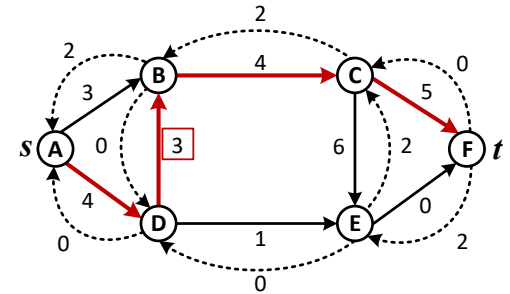
Caminho: A – B – C – E – F

Fluxo = 0 + 2 = 2



Caminho: A – D – B – C – F

Fluxo = 2





6.1. Ford-Fulkerson

Algoritmo – Exemplo 1

fordFulkerson(G, s, t)

```

1  $G_r \leftarrow G$ ;
2  $fluxoMax \leftarrow 0$ ;
3 while  $\exists$  caminho  $p$  de  $s-t$  em  $G_r$  do
4    $c_r(p) \leftarrow \min\{c_r(v, u) : (v, u) \in p\}$ ;
5   for cada arco  $(v, u) \in p$  do
6      $G_r[v][u] \leftarrow G_r[v][u] - c_r(p)$ ;
7      $G_r[u][v] \leftarrow G_r[v][u] + c_r(p)$ ;
8   end
9    $fluxoMax \leftarrow fluxoMax + c_r(p)$ ;
10 end
11 return  $fluxoMax$ 

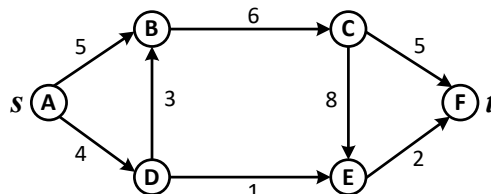
```

Atualiza as arestas e o fluxo da rede considerando $c_r(p) = 3$.

Caminho: –

Fluxo = 0

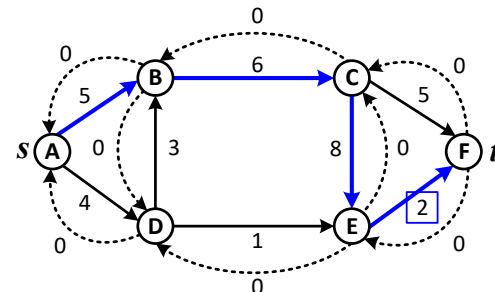
G



Caminho: A – B – C – E – F

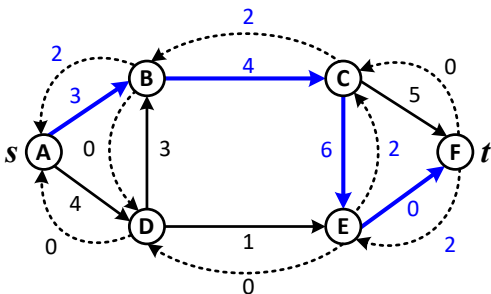
Fluxo = 0

G_r



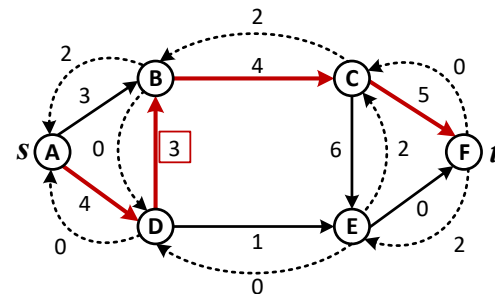
Caminho: A – B – C – E – F

Fluxo = 0 + 2 = 2



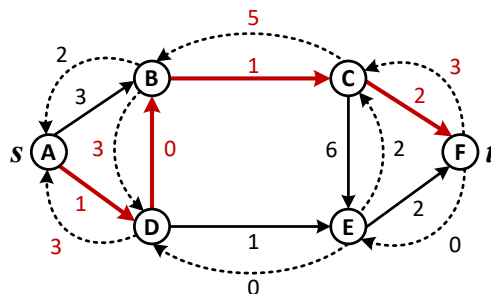
Caminho: A – D – B – C – F

Fluxo = 2



Caminho: A – D – B – C – F

Fluxo = 2 + 3 = 5





6.1. Ford-Fulkerson

Algoritmo – Exemplo 1

```
fordFulkerson( $G, s, t$ )
```

```

1  $G_r \leftarrow G$ ;
2  $fluxoMax \leftarrow 0$ ;
3 while  $\exists$  caminho  $p$  de  $s-t$  em  $G_r$  do
4    $c_r(p) \leftarrow \min\{c_r(v, u) : (v, u) \in p\}$ ;
5   for cada arco  $(v, u) \in p$  do
6      $G_r[v][u] \leftarrow G_r[v][u] - c_r(p)$ ;
7      $G_r[u][v] \leftarrow G_r[v][u] + c_r(p)$ ;
8   end
9    $fluxoMax \leftarrow fluxoMax + c_r(p)$ ;
10 end
11 return  $fluxoMax$ 

```

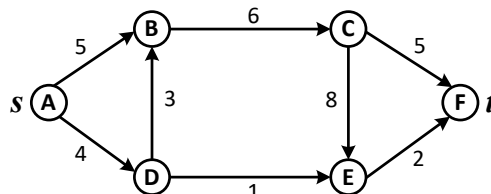
Atualiza as arestas e o fluxo da rede considerando $c_r(p) = 3$.

O algoritmo continua por outro caminho, o qual possui capacidade residual $c_r(p) = 1$.

Caminho: –

Fluxo = 0

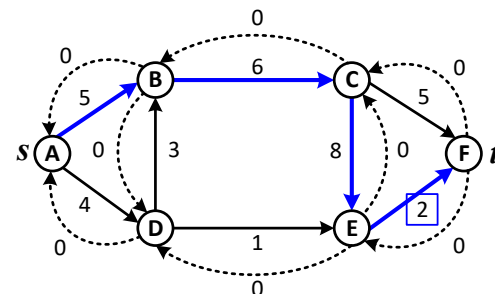
G



G_r

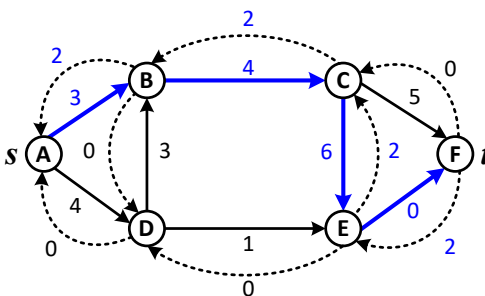
Caminho: A – B – C – E – F

Fluxo = 0



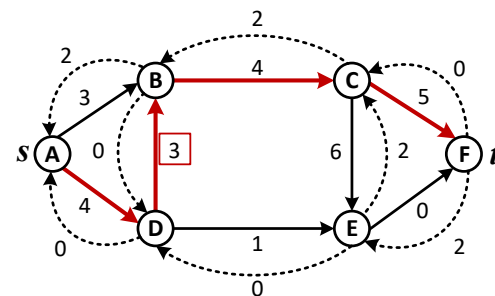
Caminho: A – B – C – E – F

Fluxo = 0 + 2 = 2



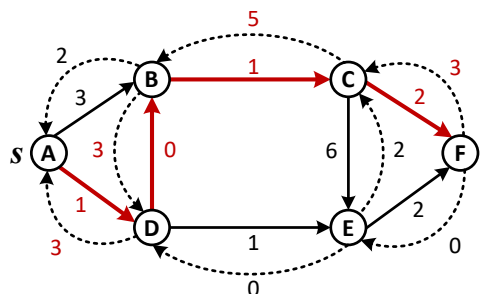
Caminho: A – D – B – C – F

Fluxo = 2



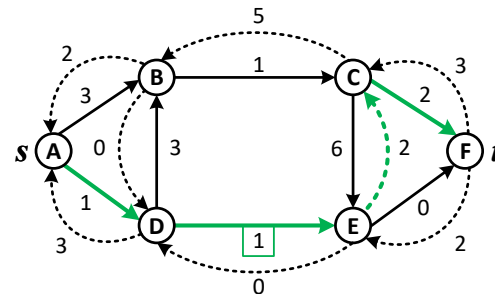
Caminho: A – D – B – C – F

Fluxo = 2 + 3 = 5



Caminho: A – D – E – C – F

Fluxo = 5





6.1. Ford-Fulkerson

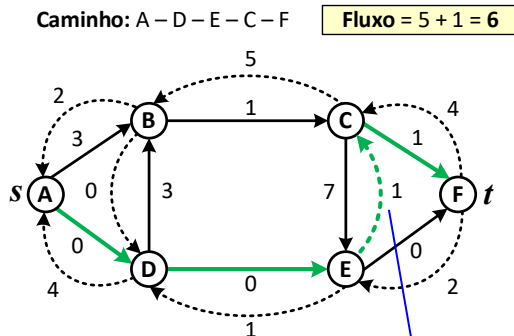
Algoritmo – Exemplo 1

fordFulkerson(G, s, t)

```

1  $G_r \leftarrow G$ ;
2  $fluxoMax \leftarrow 0$ ;
3 while  $\exists$  caminho  $p$  de  $s-t$  em  $G_r$  do
4    $c_r(p) \leftarrow \min\{c_r(v, u) : (v, u) \in p\}$ ;
5   for cada arco  $(v, u) \in p$  do
6      $G_r[v][u] \leftarrow G_r[v][u] - c_r(p)$ ;
7      $G_r[u][v] \leftarrow G_r[v][u] + c_r(p)$ ;
8   end
9    $fluxoMax \leftarrow fluxoMax + c_r(p)$ ;
10 end
11 return  $fluxoMax$ 

```



Note que a aresta
(E, C) é uma
aresta residual.

Isso significa desconsiderar a
passagem de fluxo pela aresta (C, E)
para maximizar outro caminho.

Atualiza as arestas e o fluxo da rede
considerando $c_r(p) = 3$.

O algoritmo continua por outro
caminho, o qual possui capacidade
residual $c_r(p) = 1$.

Atualiza as arestas e o fluxo da rede
considerando $c_r(p) = 1$.



6.1. Ford-Fulkerson

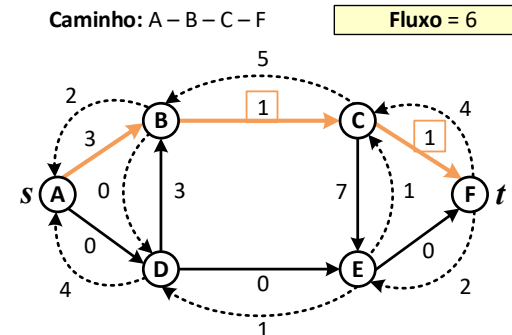
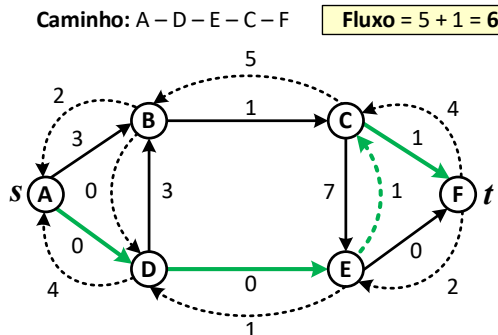
Algoritmo – Exemplo 1

fordFulkerson(G, s, t)

```

1  $G_r \leftarrow G$ ;
2  $fluxoMax \leftarrow 0$ ;
3 while  $\exists$  caminho  $p$  de  $s$ - $t$  em  $G_r$  do
4    $c_r(p) \leftarrow \min\{c_r(v, u) : (v, u) \in p\}$ ;
5   for cada arco  $(v, u) \in p$  do
6      $G_r[v][u] \leftarrow G_r[v][u] - c_r(p)$ ;
7      $G_r[u][v] \leftarrow G_r[v][u] + c_r(p)$ ;
8   end
9    $fluxoMax \leftarrow fluxoMax + c_r(p)$ ;
10 end
11 return  $fluxoMax$ 

```



O algoritmo continua por outro caminho, o qual também possui capacidade residual $c_r(p) = 1$.



6.1. Ford-Fulkerson

Algoritmo – Exemplo 1

fordFulkerson(G, s, t)

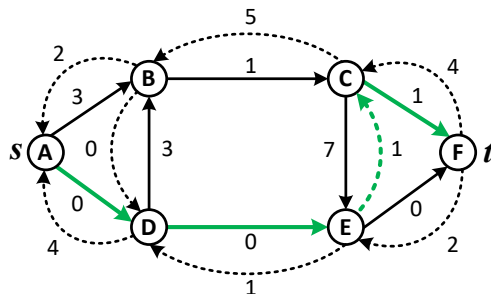
```

1  $G_r \leftarrow G$ ;
2  $fluxoMax \leftarrow 0$ ;
3 while  $\exists$  caminho  $p$  de  $s$ - $t$  em  $G_r$  do
4    $c_r(p) \leftarrow \min\{c_r(v, u) : (v, u) \in p\}$ ;
5   for cada arco  $(v, u) \in p$  do
6      $G_r[v][u] \leftarrow G_r[v][u] - c_r(p)$ ;
7      $G_r[u][v] \leftarrow G_r[u][v] + c_r(p)$ ;
8   end
9    $fluxoMax \leftarrow fluxoMax + c_r(p)$ ;
10 end
11 return  $fluxoMax$ 

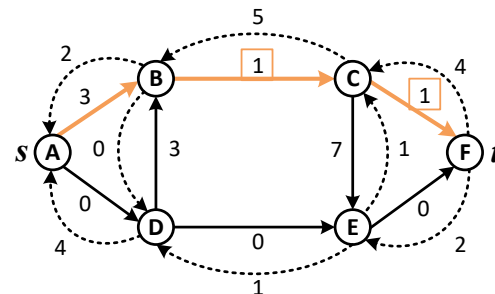
```

O algoritmo continua por outro caminho, o qual também possui capacidade residual $c_r(p) = 1$.

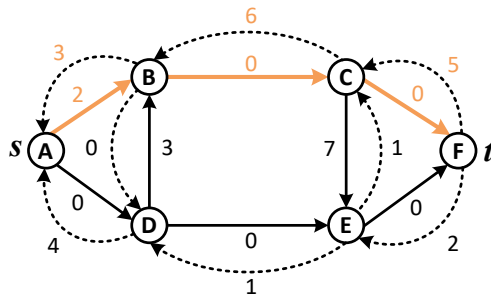
Caminho: A – D – E – C – F Fluxo = 5 + 1 = 6



Caminho: A – B – C – F Fluxo = 6



Caminho: A – B – C – F Fluxo = 6 + 1 = 7



As arestas e o fluxo são atualizados conforme a capacidade residual do caminho $c_r(p) = 1$.



6.1. Ford-Fulkerson

Algoritmo – Exemplo 1

fordFulkerson(G, s, t)

```

1  $G_r \leftarrow G$ ;
2  $fluxoMax \leftarrow 0$ ;
3 while  $\exists$  caminho  $p$  de  $s-t$  em  $G_r$  do
4    $c_r(p) \leftarrow \min\{c_r(v, u) : (v, u) \in p\}$ ;
5   for cada arco  $(v, u) \in p$  do
6      $G_r[v][u] \leftarrow G_r[v][u] - c_r(p)$ ;
7      $G_r[u][v] \leftarrow G_r[u][v] + c_r(p)$ ;
8   end
9    $fluxoMax \leftarrow fluxoMax + c_r(p)$ ;
10 end
11 return  $fluxoMax$ 

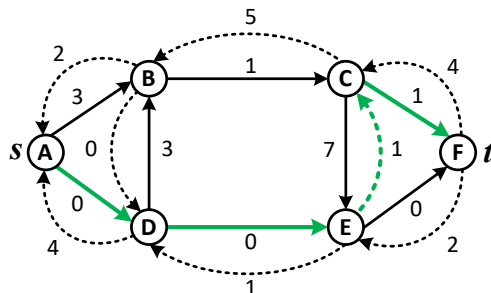
```

O algoritmo continua por outro caminho, o qual também possui capacidade residual $c_r(p) = 1$.

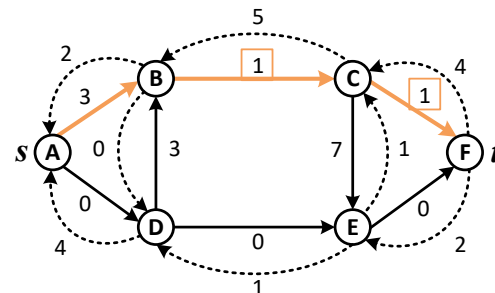
As arestas e o fluxo são atualizados conforme a capacidade residual do caminho $c_r(p) = 1$.

O método termina pois não existem mais caminhos disponíveis (ambos os fluxos de entrada de F estão saturados). O método retorna o **fluxo máximo** da rede é **7**.

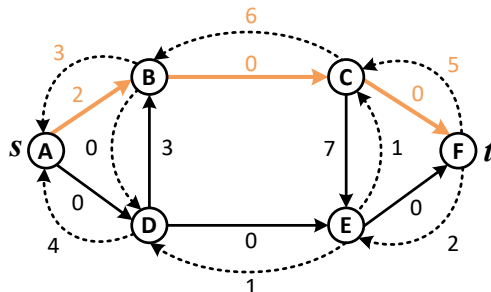
Caminho: A-D-E-C-F Fluxo = 5 + 1 = 6



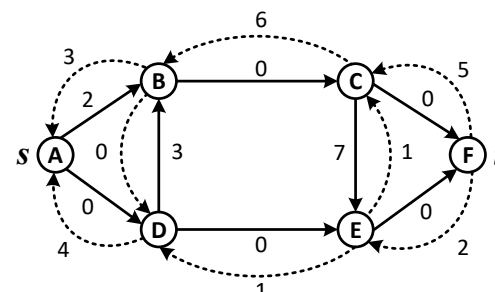
Caminho: A-B-C-F Fluxo = 6



Caminho: A-B-C-F Fluxo = 6 + 1 = 7



Fluxo Máximo = 7





6.1. Ford-Fulkerson

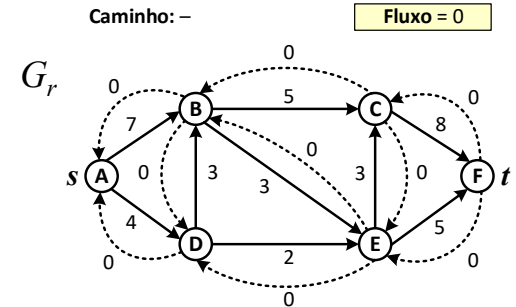
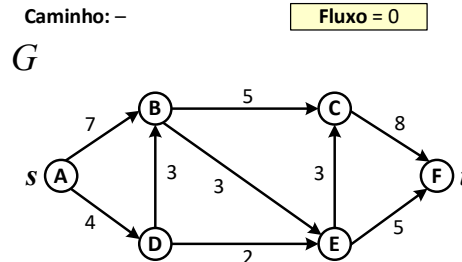
Algoritmo – Exemplo 2

fordFulkerson(G, s, t)

```

1  $G_r \leftarrow G$ ;
2  $fluxoMax \leftarrow 0$ ;
3 while  $\exists$  caminho  $p$  de  $s-t$  em  $G_r$  do
4    $c_r(p) \leftarrow \min\{c_r(v, u) : (v, u) \in p\}$ ;
5   for cada arco  $(v, u) \in p$  do
6      $G_r[v][u] \leftarrow G_r[v][u] - c_r(p)$ ;
7      $G_r[u][v] \leftarrow G_r[v][u] + c_r(p)$ ;
8   end
9    $fluxoMax \leftarrow fluxoMax + c_r(p)$ ;
10 end
11 return  $fluxoMax$ 

```



A partir de uma rede de fluxo G é obtida uma rede residual G_r e inicializada a variável $fluxoMax$.



6.1. Ford-Fulkerson

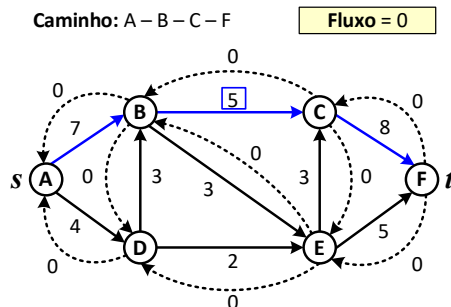
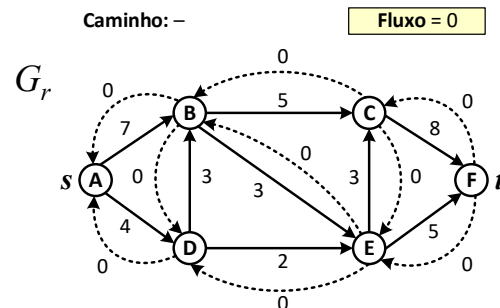
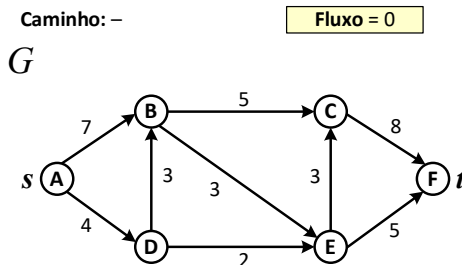
Algoritmo – Exemplo 2

fordFulkerson(G, s, t)

```

1  $G_r \leftarrow G$ ;
2  $fluxoMax \leftarrow 0$ ;
3 while  $\exists$  caminho  $p$  de  $s-t$  em  $G_r$  do
4    $c_r(p) \leftarrow \min\{c_r(v, u) : (v, u) \in p\}$ ;
5   for cada arco  $(v, u) \in p$  do
6      $G_r[v][u] \leftarrow G_r[v][u] - c_r(p)$ ;
7      $G_r[u][v] \leftarrow G_r[v][u] + c_r(p)$ ;
8   end
9    $fluxoMax \leftarrow fluxoMax + c_r(p)$ ;
10 end
11 return  $fluxoMax$ 

```



A partir de uma rede de fluxo G é obtida uma rede residual G_r e inicializada a variável $fluxoMax$.

O método continua identificando um caminho p (A – F), cuja capacidade residual é $c_r(p) = 5$.



6.1. Ford-Fulkerson

Algoritmo – Exemplo 2

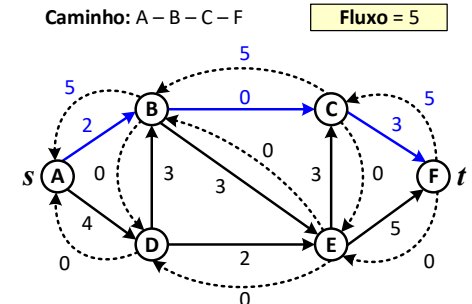
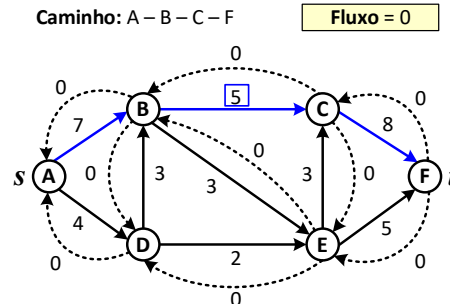
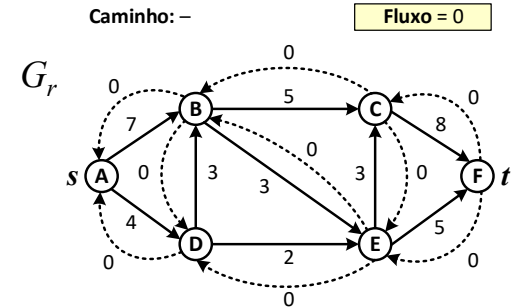
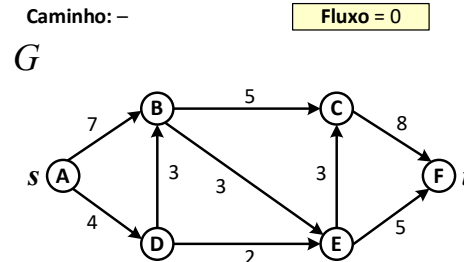
```
fordFulkerson( $G, s, t$ )
```

```

1  $G_r \leftarrow G$ ;
2  $fluxoMax \leftarrow 0$ ;
3 while  $\exists$  caminho  $p$  de  $s-t$  em  $G_r$  do
4    $c_r(p) \leftarrow \min\{c_r(v, u) : (v, u) \in p\}$ ;
5   for cada arco  $(v, u) \in p$  do
6      $G_r[v][u] \leftarrow G_r[v][u] - c_r(p)$ ;
7      $G_r[u][v] \leftarrow G_r[u][v] + c_r(p)$ ;
8   end
9    $fluxoMax \leftarrow fluxoMax + c_r(p)$ ;
10 end
11 return  $fluxoMax$ 

```

As arestas originais e residuais do caminho p em G_r são atualizadas conforme $c_r(p) = 5$, além do fluxo.





6.1. Ford-Fulkerson

Algoritmo – Exemplo 2

```
fordFulkerson( $G, s, t$ )
```

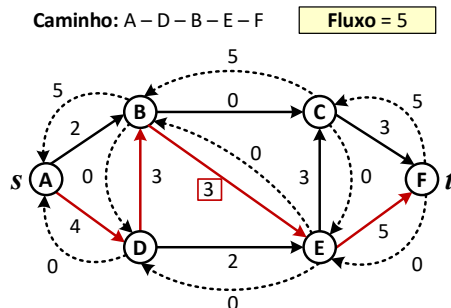
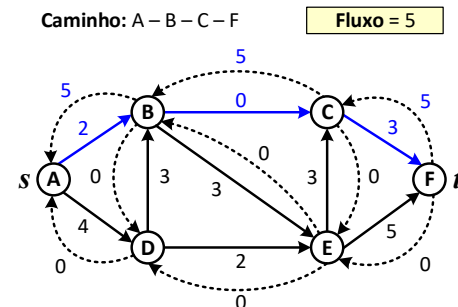
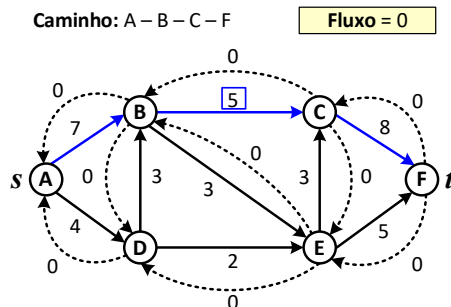
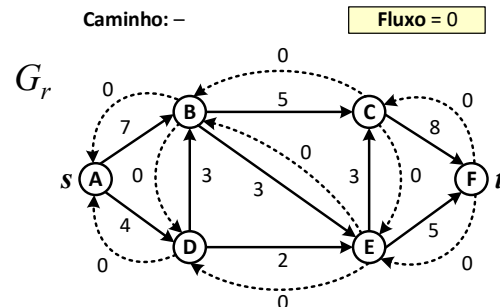
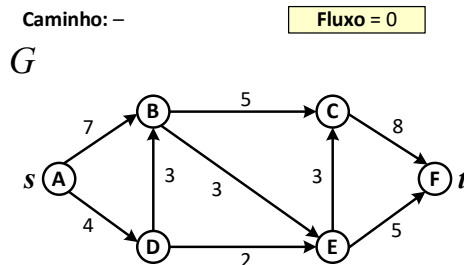
```

1  $G_r \leftarrow G$ ;
2  $fluxoMax \leftarrow 0$ ;
3 while  $\exists$  caminho  $p$  de  $s-t$  em  $G_r$  do
4    $c_r(p) \leftarrow \min\{c_r(v, u) : (v, u) \in p\}$ ;
5   for cada arco  $(v, u) \in p$  do
6      $G_r[v][u] \leftarrow G_r[v][u] - c_r(p)$ ;
7      $G_r[u][v] \leftarrow G_r[u][v] + c_r(p)$ ;
8   end
9    $fluxoMax \leftarrow fluxoMax + c_r(p)$ ;
10 end
11 return  $fluxoMax$ 

```

As arestas originais e residuais do caminho p em G_r são atualizadas conforme $c_r(p) = 5$, além do fluxo.

O método considera outro caminho p , cuja capacidade residual é $c_r(p) = 3$.





6.1. Ford-Fulkerson

Algoritmo – Exemplo 2

```
fordFulkerson( $G, s, t$ )
```

```

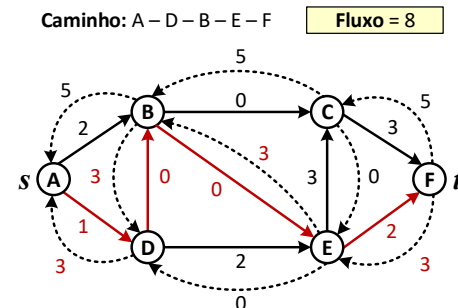
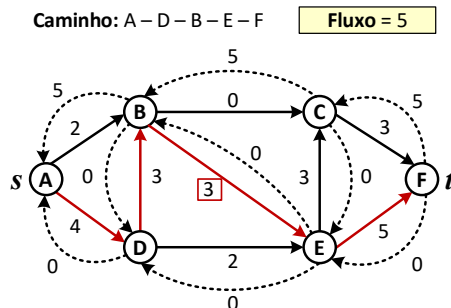
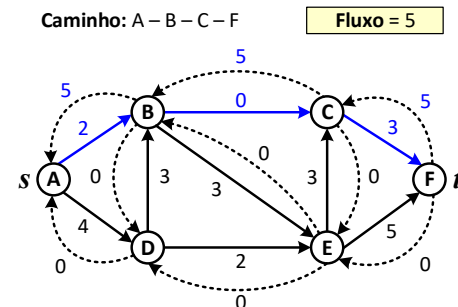
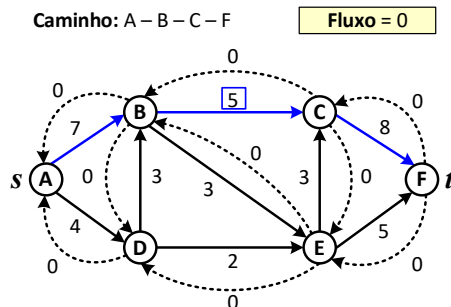
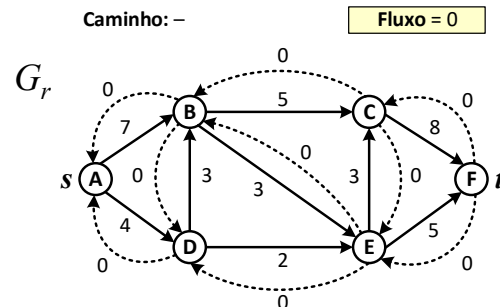
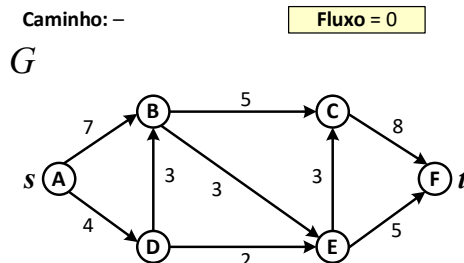
1  $G_r \leftarrow G$ ;
2  $fluxoMax \leftarrow 0$ ;
3 while  $\exists$  caminho  $p$  de  $s$ - $t$  em  $G_r$  do
4    $c_r(p) \leftarrow \min\{c_r(v, u) : (v, u) \in p\}$ ;
5   for cada arco  $(v, u) \in p$  do
6      $G_r[v][u] \leftarrow G_r[v][u] - c_r(p)$ ;
7      $G_r[u][v] \leftarrow G_r[u][v] + c_r(p)$ ;
8   end
9    $fluxoMax \leftarrow fluxoMax + c_r(p)$ ;
10 end
11 return  $fluxoMax$ 

```

As arestas originais e residuais do caminho p em G_r são atualizadas conforme $c_r(p) = 5$, além do fluxo.

O método considera outro caminho p , cuja capacidade residual é $c_r(p) = 3$.

As arestas do caminho p são atualizadas, além do Fluxo Máximo.





6.1. Ford-Fulkerson

Algoritmo – Exemplo 2

fordFulkerson(G, s, t)

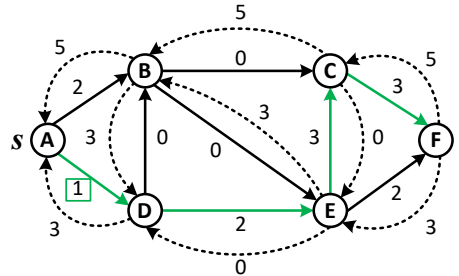
```

1  $G_r \leftarrow G$ ;
2  $fluxoMax \leftarrow 0$ ;
3 while  $\exists$  caminho  $p$  de  $s-t$  em  $G_r$  do
4    $c_r(p) \leftarrow \min\{c_r(v, u) : (v, u) \in p\}$ ;
5   for cada arco  $(v, u) \in p$  do
6      $G_r[v][u] \leftarrow G_r[v][u] - c_r(p)$ ;
7      $G_r[u][v] \leftarrow G_r[v][u] + c_r(p)$ ;
8   end
9    $fluxoMax \leftarrow fluxoMax + c_r(p)$ ;
10 end
11 return  $fluxoMax$ 

```

Caminho: A – D – E – C – F

Fluxo = 8



O algoritmo continua por outro caminho, o qual possui capacidade residual $c_r(p) = 1$.



6.1. Ford-Fulkerson

Algoritmo – Exemplo 2

```
fordFulkerson( $G, s, t$ )
```

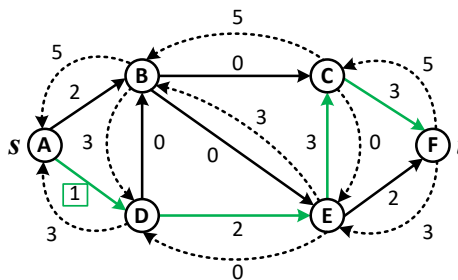
```

1  $G_r \leftarrow G$ ;
2  $fluxoMax \leftarrow 0$ ;
3 while  $\exists$  caminho  $p$  de  $s-t$  em  $G_r$  do
4    $c_r(p) \leftarrow \min\{c_r(v, u) : (v, u) \in p\}$ ;
5   for cada arco  $(v, u) \in p$  do
6      $G_r[v][u] \leftarrow G_r[v][u] - c_r(p)$ ;
7      $G_r[u][v] \leftarrow G_r[v][u] + c_r(p)$ ;
8   end
9    $fluxoMax \leftarrow fluxoMax + c_r(p)$ ;
10 end
11 return  $fluxoMax$ 

```

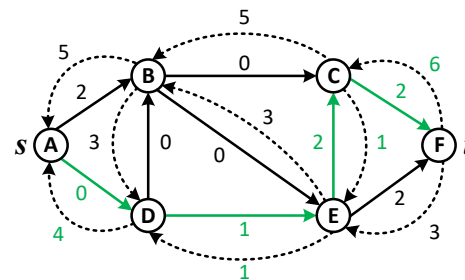
Caminho: A – D – E – C – F

Fluxo = 8



Caminho: A – D – E – C – F

Fluxo = 9



O algoritmo continua por outro caminho, o qual possui capacidade residual $c_r(p) = 1$.

As capacidades das arestas do caminho e o fluxo são atualizados.



6.1. Ford-Fulkerson

Algoritmo – Exemplo 2

```
fordFulkerson( $G, s, t$ )
```

```

1  $G_r \leftarrow G$ ;
2  $fluxoMax \leftarrow 0$ ;
3 while  $\exists$  caminho  $p$  de  $s$ - $t$  em  $G_r$  do
4    $c_r(p) \leftarrow \min\{c_r(v, u) : (v, u) \in p\}$ ;
5   for cada arco  $(v, u) \in p$  do
6      $G_r[v][u] \leftarrow G_r[v][u] - c_r(p)$ ;
7      $G_r[u][v] \leftarrow G_r[u][v] + c_r(p)$ ;
8   end
9    $fluxoMax \leftarrow fluxoMax + c_r(p)$ ;
10 end
11 return  $fluxoMax$ 

```

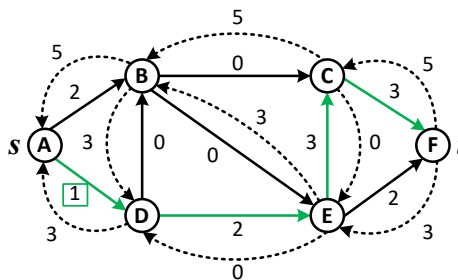
O algoritmo continua por outro caminho, o qual possui capacidade residual $c_r(p) = 1$.

As capacidades das arestas do caminho são atualizadas.

Outro caminho é analisado. Note que ele contém uma aresta residual (B, D) na sua constituição.

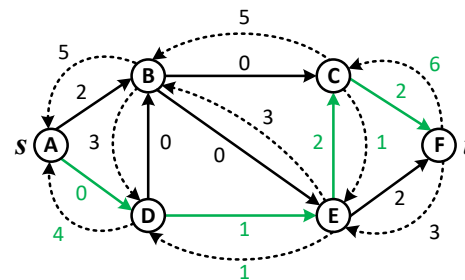
Caminho: A – D – E – C – F

Fluxo = 8



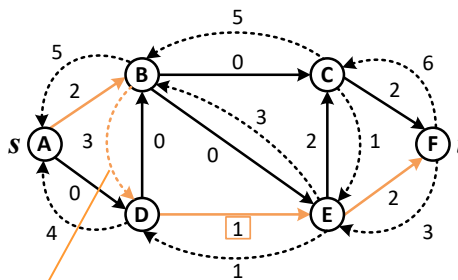
Caminho: A – D – E – C – F

Fluxo = 9



Caminho: A – B – D – E – F

Fluxo = 9



Isso significa desconsiderar a passagem de fluxo pela (D, B) para maximizar outro caminho.



6.1. Ford-Fulkerson

Algoritmo – Exemplo 2

fordFulkerson(G, s, t)

```

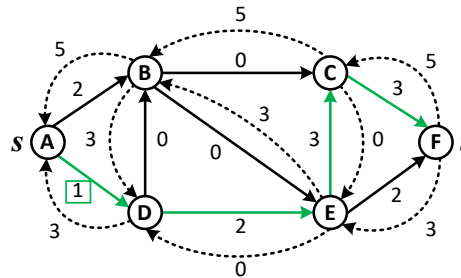
1  $G_r \leftarrow G$ ;
2  $fluxoMax \leftarrow 0$ ;
3 while  $\exists$  caminho  $p$  de  $s$ – $t$  em  $G_r$  do
4    $c_r(p) \leftarrow \min\{c_r(v, u) : (v, u) \in p\}$ ;
5   for cada arco  $(v, u) \in p$  do
6      $G_r[v][u] \leftarrow G_r[v][u] - c_r(p)$ ;
7      $G_r[u][v] \leftarrow G_r[v][u] + c_r(p)$ ;
8   end
9    $fluxoMax \leftarrow fluxoMax + c_r(p)$ ;
10 end
11 return  $fluxoMax$ 

```

As arestas originais e residuais são atualizadas conforme a capacidade residual $c_r(p) = 1$, além do Fluxo.

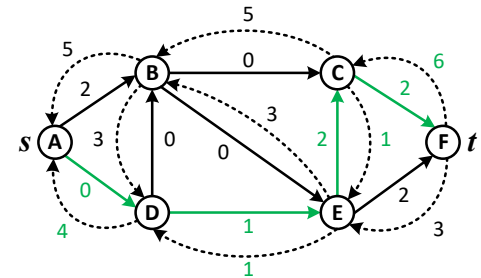
Caminho: A–D–E–C–F

Fluxo = 8



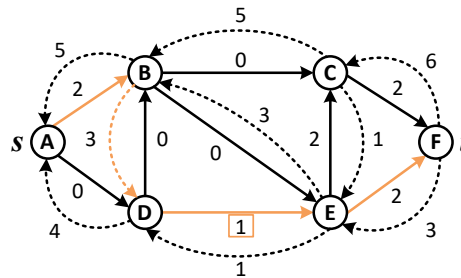
Caminho: A–D–E–C–F

Fluxo = 9



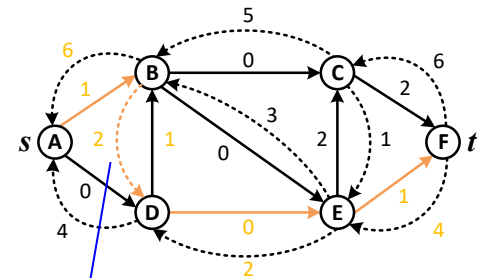
Caminho: A–B–D–E–F

Fluxo = 9



Caminho: A–D–E–C–F

Fluxo = 10



aresta residual



6.1. Ford-Fulkerson

Algoritmo – Exemplo 2

fordFulkerson(G, s, t)

```

1  $G_r \leftarrow G$ ;
2  $fluxoMax \leftarrow 0$ ;
3 while  $\exists$  caminho  $p$  de  $s$ - $t$  em  $G_r$  do
4    $c_r(p) \leftarrow \min\{c_r(v, u) : (v, u) \in p\}$ ;
5   for cada arco  $(v, u) \in p$  do
6      $G_r[v][u] \leftarrow G_r[v][u] - c_r(p)$ ;
7      $G_r[u][v] \leftarrow G_r[v][u] + c_r(p)$ ;
8   end
9    $fluxoMax \leftarrow fluxoMax + c_r(p)$ ;
10 end
11 return  $fluxoMax$ 

```

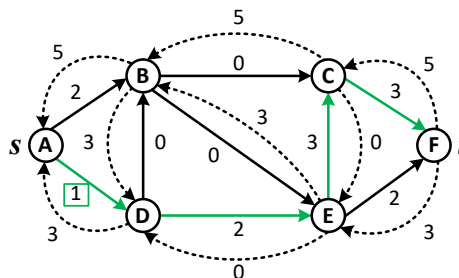
As arestas originais e residuais da rede são atualizadas conforme a capacidade residual $c_r(p) = 1$.

O método termina quando não existirem mais caminhos alternativos de A para F.

O **Fluxo Máximo** da rede é **10**.

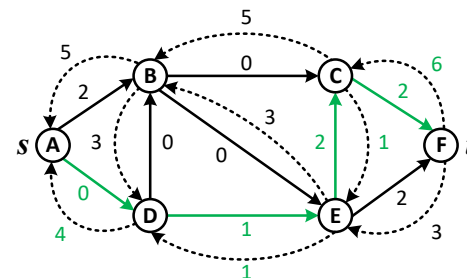
Caminho: A – D – E – C – F

Fluxo = 8



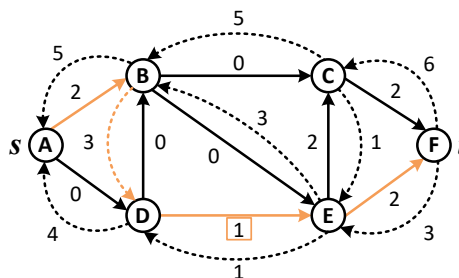
Caminho: A – D – E – C – F

Fluxo = 9



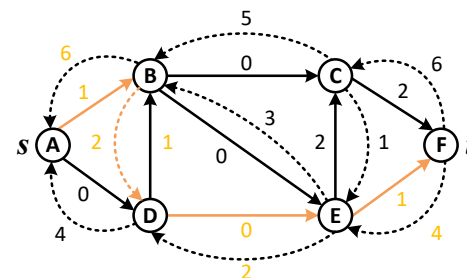
Caminho: A – B – D – E – F

Fluxo = 9

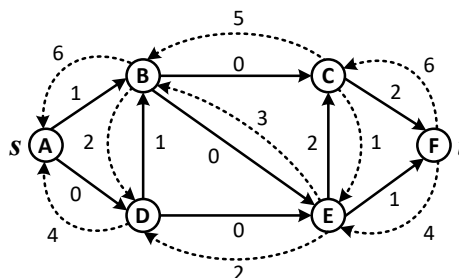


Caminho: A – D – E – C – F

Fluxo = 10



Fluxo Máximo = 10



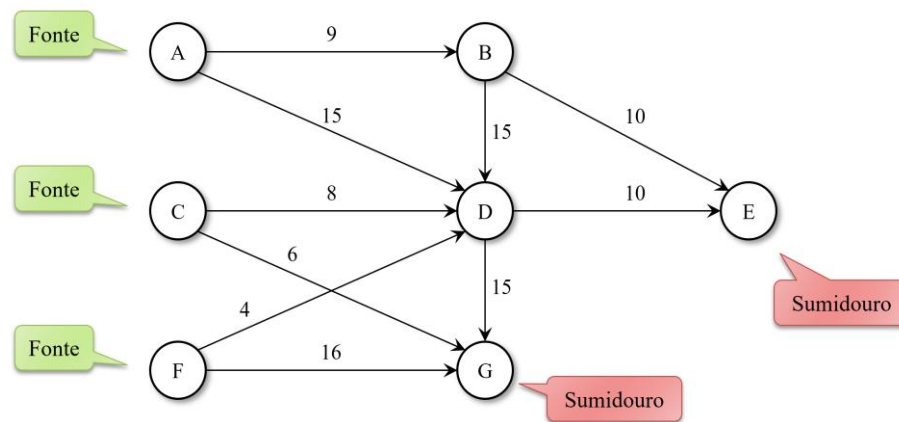


6.1. Ford-Fulkerson

Casos Especiais

Diversos problemas podem ser resolvidos como um problema de fluxo máximo, mas alguns deles parecem não se adequar a todas as restrições colocadas até então.

Em diversos destes casos é possível adequar a modelagem conforme um problema de fluxo máximo, apenas representando o grafo de forma conveniente.





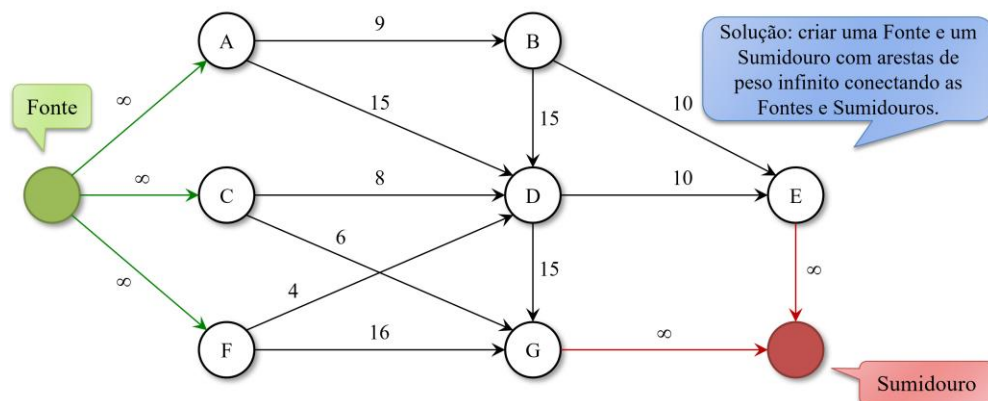
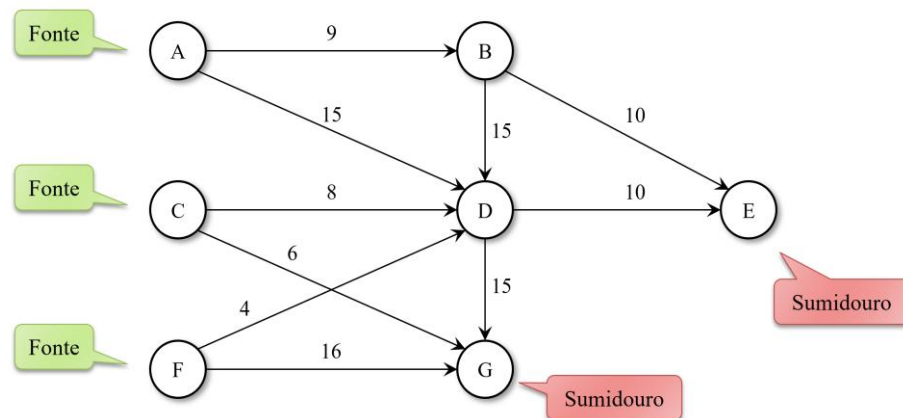
6.1. Ford-Fulkerson

Casos Especiais

Diversos problemas podem ser resolvidos como um problema de fluxo máximo, mas alguns deles parecem não se adequar a todas as restrições colocadas até então.

Em diversos destes casos é possível adequar a modelagem conforme um problema de fluxo máximo, apenas representando o grafo de forma conveniente.

Um caso especial é quando existem **múltiplas fontes ou sumidouros**. A solução é incluir um vértice fonte e um sumidouro com as arestas de peso infinito conectando as fontes e sumidouros originais.



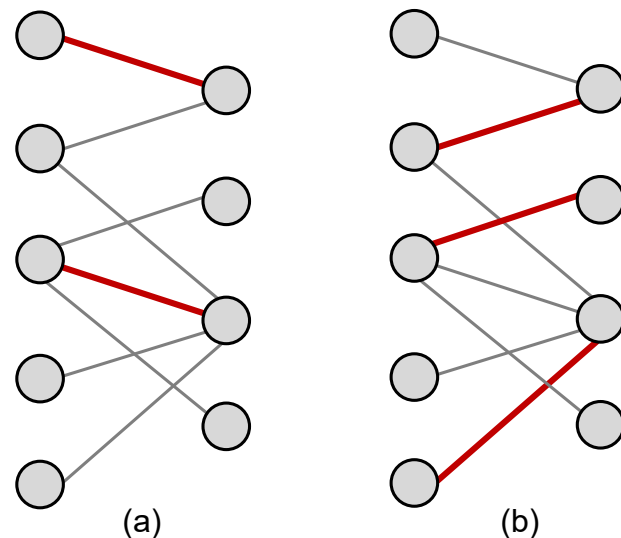


6.2. Emparelhamento em Grafos Bipartidos

Redes de Fluxo com Múltiplas Origens e Sorvedouros

Redes de fluxo com múltiplas origens e sorvedouros podem ser utilizadas para solução do problema de Emparelhamento Máximo em Grafo Bipartido.

Dado um grafo não orientado $G = (V, E)$, um emparelhamento (*matching*) é um subconjunto de arestas $M \subseteq E$ tais que, para todos os vértices $v \in V$, no máximo uma aresta de M é incidente sobre v .



Exemplos de emparelhamento com cardinalidade 2 (a) e em (b) com cardinalidade 3 (máximo).



6.2. Emparelhamento em Grafos Bipartidos

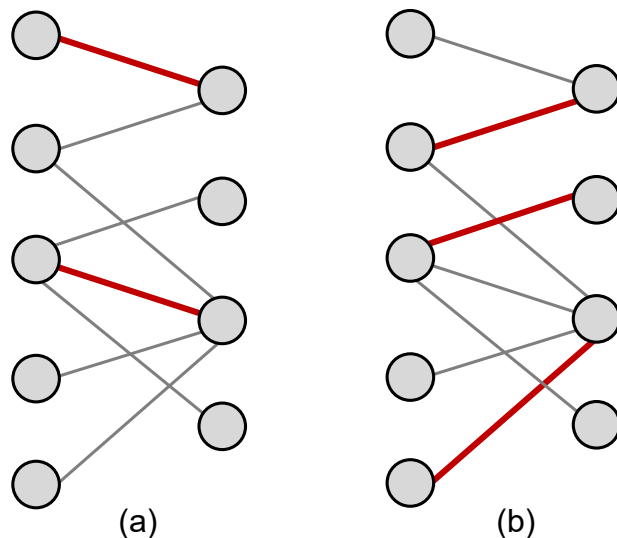
Redes de Fluxo com Múltiplas Origens e Sorvedouros

Redes de fluxo com múltiplas origens e sorvedouros podem ser utilizados para solução do problema de Emparelhamento Máximo em Grafo Bipartido.

Dado um grafo não orientado $G = (V, E)$, um emparelhamento (*matching*) é um subconjunto de arestas $M \subseteq E$ tais que, para todos os vértices $v \in V$, no máximo uma aresta de M é incidente sobre v .

Um vértice $v \in V$ é dito **correspondido** pelo emparelhamento se incide alguma aresta de M sobre ele, ou não-correspondido caso contrário.

Um **emparelhamento máximo** é aquele que possui a maior cardinalidade, ou seja, é um emparelhamento M tal que, para qualquer emparelhamento M' temos $|M| \geq |M'|$



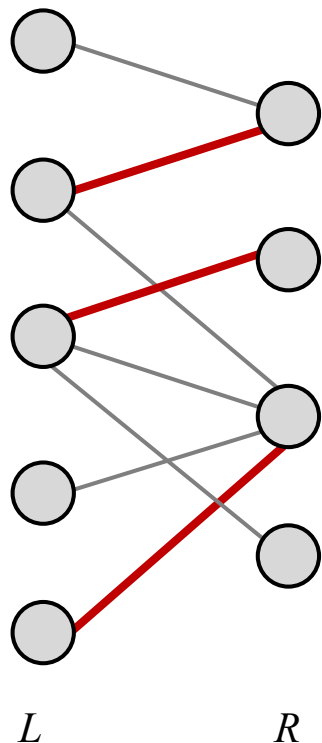
Exemplos de emparelhamento com cardinalidade 2 (a) e em (b) com cardinalidade 3 (máximo).



6.2. Emparelhamento em Grafos Bipartidos

Emparelhamento Máximo em grafo Bipartido

Possui aplicações práticas como, por exemplo, a alocação de tarefas a máquinas. Existe um conjunto de máquinas (L) e um conjunto de tarefas (R) a serem executadas simultaneamente.



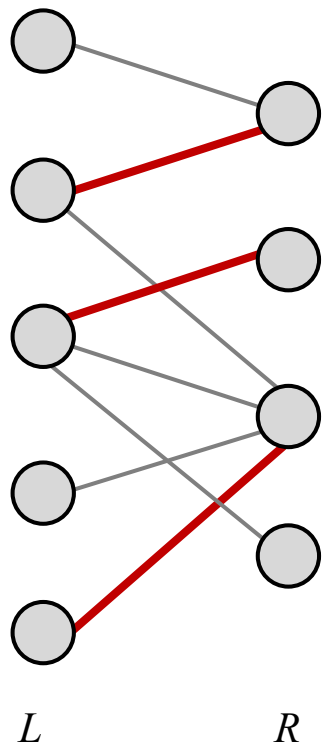
Considera-se que uma aresta $(v, u) \in E$ indica que uma determinada máquina $v \in L$ é capaz de executar uma dada tarefa $u \in R$. O emparelhamento máximo fornece trabalho para a maior quantidade de máquinas possível.



6.2. Emparelhamento em Grafos Bipartidos

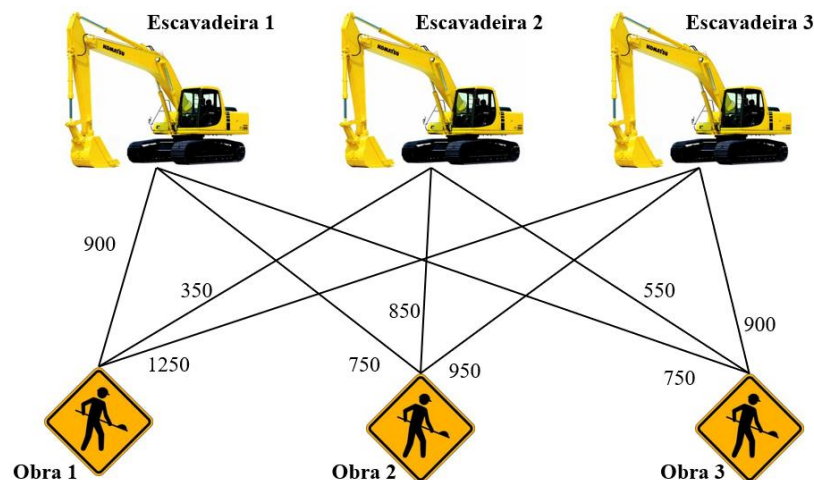
Emparelhamento Máximo em grafo Bipartido

Possui aplicações práticas como, por exemplo, a alocação de tarefas a máquinas. Existe um conjunto de máquinas (L) e um conjunto de tarefas (R) a serem executadas simultaneamente.



Considera-se que uma aresta $(v, u) \in E$ indica que uma determinada máquina $v \in L$ é capaz de executar uma dada tarefa $u \in R$. O emparelhamento máximo fornece trabalho para a maior quantidade de máquinas possível.

Ex. considere escavadeiras que estão em garagens diferentes precisam ser enviadas até o local de uma obra. O objetivo é alocar o máximo possível de máquinas e minimizar o custo de transporte.



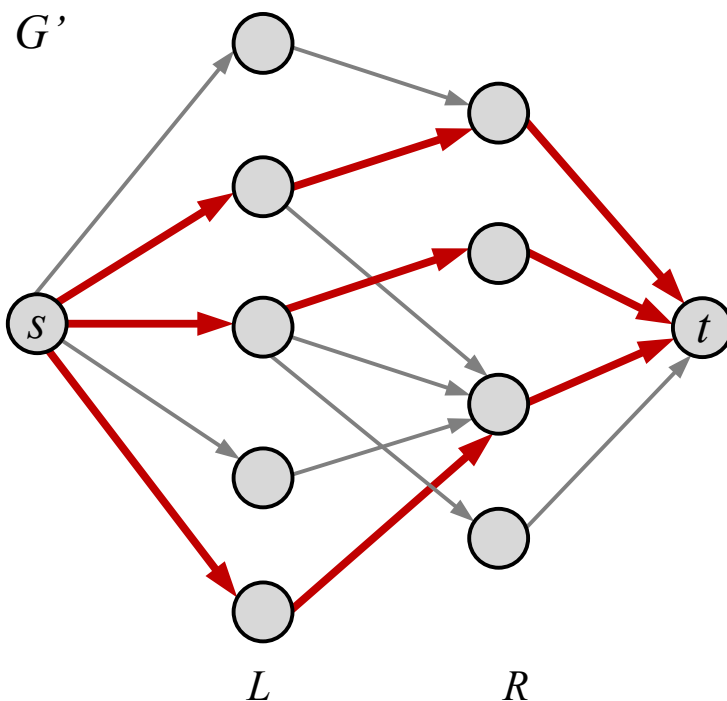
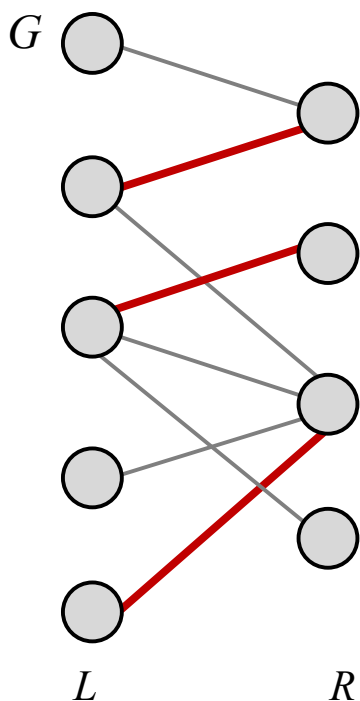


6.2. Emparelhamento em Grafos Bipartidos

Emparelhamento Máximo em grafo Bipartido

O método de Ford-Fulkerson pode ser utilizado para encontrar um emparelhamento máximo em um grafo bipartido não orientado $G = (V, E)$. A ideia é construir um fluxo em rede na qual os fluxos equivalem a emparelhamentos.

Define-se um fluxo em rede correspondente $G' = (V', E')$ para o grafo G .



São adicionados vértices origem e sorvedouro. As arestas em vermelho têm capacidade unitária, e as demais não conduzem fluxo (igual a 0).

As arestas em vermelho equivalem as arestas de um emparelhamento máximo em grafo bipartido.



6.2. Emparelhamento em Grafos Bipartidos

Algoritmo Húngaro

Publicado em 1955 por Harold W. Kuhn no artigo “*The Hungarian Method for the Assignment Problem*”. Baseado nos estudos dos autores húngaros Dénes König (emparelhamentos e coberturas em grafos bipartidos), e de Jenő Egerváry (emparelhamentos máximos e minimização linear). Posteriormente, James Munkres refinou o método e estabeleceu sua implementação com complexidade polinomial (algoritmo de Kuhn–Munkres).

Princípio de Funcionamento: Se for subtraída a mesma constante de todos os elementos de uma linha (ou coluna) de uma matriz de custos, a solução ótima do problema de atribuição não se altera.

Quando uma linha ou coluna já contém um zero, isso significa que aquela linha/coluna já está "otimizada" em relação aos custos. A redução deixa os valores inalterados.

Após aplicar as etapas de redução de linhas e colunas a matriz de custos reduzida contém um número suficiente de zeros para permitir a formação de uma atribuição ótima, ou seja, selecionar n zeros independentes (um em cada linha e um em cada coluna) que formem uma permutação válida.



6.2. Algoritmo Húngaro

Aplicações

- **Alocação de Recursos em Manufatura e Produção:** Atribuição de operários a máquinas, designação de tarefas a processadores, escalonamento de jobs em fábricas inteligentes, otimização de linhas de produção, minimização de tempo de setup.
- **Logística e Distribuição:** Atribuição de motoristas a rotas de entrega, alocação de veículos a clientes, otimização de frotas, designação de centros de distribuição a pontos de venda, minimização de custos de transporte;
- **Visão Computacional e Rastreamento de Objetos:** Correspondência de detecções em frames consecutivos (Multi-Object Tracking), fusão de sensores (LiDAR + câmera), alinhamento de características em imagens, reconhecimento facial com múltiplas detecções, reconstrução 3D;
- **Bioinformática e Análise Genômica:** Alinhamento de sequências de DNA/proteínas, emparelhamento de genes homólogos entre espécies, análise de estruturas moleculares, identificação de correspondências em análise de genomas;
- **Telecomunicações e Redes:** Alocação de frequências de rádio a torres de transmissão, atribuição de canais a usuários, roteamento de pacotes em redes, alocação de time-slots em redes 5G, minimização de interferência espectral;
- **Gestão de Projetos e Recursos Humanos:** Alocação de membros da equipe a tarefas, designação de consultores a clientes, distribuição de responsabilidades em projetos, otimização de utilização de recursos humanos;



6.2. Algoritmo Húngaro

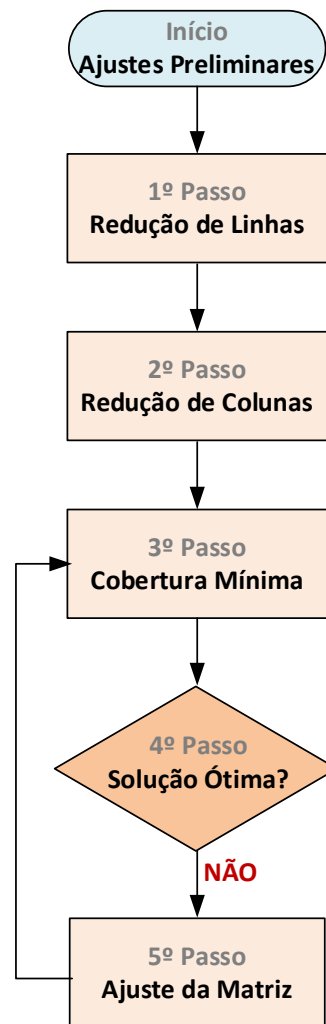
Aplicações

- **Sistemas de Recomendação e Matching:** Emparelhamento de usuários a produtos (e-commerce), matching em plataformas de economia compartilhada (Uber, Airbnb), formação de equipes com habilidades complementares, pareamento de credores e devedores em mercados de crédito.
- **Otimização de Layout e Design:** Posicionamento de componentes em circuitos VLSI, arranjo de máquinas em fábricas, distribuição de serviços públicos em cidades, planejamento urbano, minimização de comprimento de fios e interferências.
- **Problemas Financeiros e de Investimento:** Alocação de capital a projetos ou ativos, distribuição de orçamento entre departamentos, otimização de portfólio de investimentos, matching de credores a devedores, alocação de recursos em startups.
- **Análise de Redes Sociais e Colaborativas:** Detecção de comunidades em grafos bipartidos, emparelhamento de colaboradores em projetos, identificação de pares em redes de coautoria, análise de estruturas de colaboração em pesquisa.
- **Problemas de Transporte e Fluxo:** Problema de transporte balanceado (origens a destinos), alocação de mercadorias em centros de distribuição, otimização de rotas de transbordo, minimização de custos logísticos.
- **Reconhecimento de Padrões e Machine Learning:** Correspondência de features entre imagens, alinhamento de modelos em visão 3D, matching de padrões em séries temporais, associação de clusters em análise de dados.



6.2. Algoritmo Húngaro

Procedimento



Início – Se necessário, realizar ajustes na matriz de custos se ela estiver desbalanceada ou se o problema for de maximização (detalhes no Exemplo 3).

1º Passo – Redução de Linhas: consiste em identificar o menor valor de cada **linha** da matriz de custos e subtraí-lo de todos os valores da própria linha.

$$C'[i, j] = C[i, j] - \min(C[i, :]) \quad \text{Ex. } [10, 3, 5, 4] \rightarrow [7, 0, 2, 1]$$

2º Passo – Redução de Colunas: identificar o menor valor de cada **coluna** e subtraí-lo de todos os valores da própria coluna. $C''[i, j] = C'[i, j] - \min(C'[:, j])$

3º Passo – Cobertura Mínima: obter o número mínimo de riscos de linhas ou colunas que cobrem todos os valores zero da matriz de custos C'' .

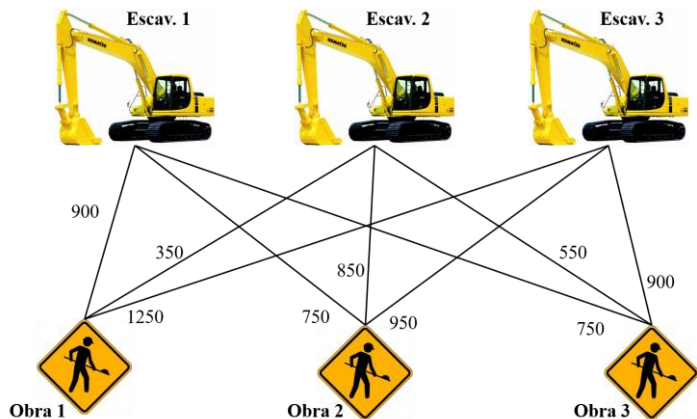
4º Passo – Teste de Solução Ótima: Se o número mínimo de riscos necessários para cobrir todos os zeros é igual a ordem da matriz, então uma alocação ótima foi encontrada e o procedimento encerra, retornando o custo total e o emparelhamento. Caso contrário, executa o 5º Passo.

5º Passo – Ajuste da Matriz: Identifica o menor valor da matriz ainda não riscado. Subtrair esse valor de todos os valores não riscados. Nos valores na interseção de linhas e colunas riscadas deve-se somar com o menor valor. Retorna ao 3º Passo.



6.2. Algoritmo Húngaro

Exemplo 1 – Alocação de Máquinas



Uma construtora possui três garagens e cada qual possui uma escavadeira. As escavadeiras devem ser transportadas para três obras distintas e o custo do transporte para cada obra é dado pelo grafo ao lado. Qual escavadeira deverá ser alocada a cada obra para minimizar o custo?

MÁQUINA	OBRA		
	1	2	3
1	900	750	750
2	350	850	550
3	1.250	950	900

Matriz de Custos – Relaciona o custo do transporte de cada máquina para cada obra. **Obs.** Matriz de Custo é diferente da Matriz de Adjacências.

MÁQUINA	OBRA		
	1	2	3
1	150	0	0
2	0	500	200
3	350	50	0

1º Passo – Subtrair 750 dos valores da 1ª linha, 350 dos valores da 2ª linha e 900 dos valores da 3ª linha.



6.2. Algoritmo Húngaro

Exemplo 1 – Alocação de Máquinas

	OBRA		
MÁQUINA	1	2	3
1	150	0	0
2	0	500	200
3	350	50	0

2º Passo – Subtrair o menor valor de cada coluna de todos os valores da mesma coluna. Neste exemplo, a matriz permanece inalterada, pois o menor valor em cada coluna é 0.

	OBRA		
MÁQUINA	1	2	3
1	150	0	0
2	0	500	200
3	350	50	0

3º Passo – Riscar as linhas e colunas de tal modo que todas as entradas zero da matriz de custo sejam riscadas, utilizando um número mínimo de riscos.

	OBRA		
MÁQUINA	1	2	3
1	150	0	0
2	0	500	200
3	350	50	0

4º Passo – O número mínimo de riscos para cobrir todos os zeros da matriz é três (igual a sua ordem). Logo, existem três zeros, um em cada linha e em cada coluna da matriz, que corresponde à alocação ótima.

	OBRA		
MÁQUINA	1	2	3
1	900	750	750
2	350	850	550
3	1.250	950	900

Resultado – A Escavadeira 1 será alocada na Obra 2; a 2 na Obra 1 e a 3 na Obra 3, ocasionando no custo mínimo de:

$$R\$ 350,00 + R\$ 750,00 + R\$ 900,00 = R\$ 2.000,00.$$



6.2. Algoritmo Húngaro

Exemplo 2 – Atribuição de Corretores

Uma empresa especializada em avaliação imobiliária precisa designar seus corretores para visitar imóveis de clientes e estimar seu valor de mercado. Cada corretor inicia o dia em uma localização específica, e cada imóvel deve ser visitado por exatamente um corretor. Como o deslocamento representa um custo operacional, a empresa deseja definir a atribuição dos corretores aos imóveis de forma que a soma das distâncias percorridas seja a menor possível.

CORRETOR	IMÓVEL		
	1	2	3
1	10	15	9
2	9	18	5
3	6	14	3

CORRETOR	IMÓVEL		
	1	2	3
1	1	6	0
2	4	13	0
3	3	11	0

Matriz de Custos – Relaciona o custo (distância em km) do deslocamento do corretor até um dado imóvel.

1º Passo – Em cada linha da matriz identifica-se o menor elemento e subtrai-se esse valor de todos os elementos da mesma linha. Ex. 1ª linha o menor custo é 9, todos elementos da linha são subtraídos desse valor.



6.2. Algoritmo Húngaro

Exemplo 2 – Atribuição de Corretores

CORRETOR	IMÓVEL		
	1	2	3
1	10	15	9
2	9	18	5
3	6	14	3

Matriz de Custos – Relaciona o custo (distância em km) do deslocamento do corretor até um dado imóvel.

CORRETOR	IMÓVEL		
	1	2	3
1	1	6	0
2	4	13	0
3	3	11	0

1º Passo – Em cada linha da matriz identifica-se o menor elemento e subtrai-se esse valor de todos os elementos da mesma linha. Ex. 1ª linha o menor custo é 9, todos elementos da linha são subtraídos desse valor.

CORRETOR	IMÓVEL		
	1	2	3
1	0	0	0
2	3	7	0
3	2	5	0

2º Passo – Em cada coluna da matriz identifica-se o menor elemento e subtrai-se esse valor de todos os elementos da mesma coluna. Ex. 2ª coluna o menor custo é 6.



6.2. Algoritmo Húngaro

Exemplo 2 – Atribuição de Corretores

CORRETOR	IMÓVEL		
	1	2	3
1	0	0	0
2	3	7	0
3	2	5	0

3º Passo – Riscar as linhas e colunas de tal modo que todos os valores zero da matriz de custo sejam riscados, utilizando a menor quantidade de riscos possível. Em seguida, verificar o número de riscos necessários.

Se número de riscos é menor que a ordem da matriz, ela deverá ser novamente reduzida. No exemplo, foram necessários 2 riscos para cobrir todos os zeros, mas a ordem da matriz é 3.



6.2. Algoritmo Húngaro

Exemplo 2 – Atribuição de Corretores

CORRETOR	IMÓVEL		
	1	2	3
1	0	0	0
2	3	7	0
3	2	5	0

menor valor

3º Passo – Riscar as linhas e colunas de tal modo que todos os valores zero da matriz de custo sejam riscados, utilizando a menor quantidade de riscos possível. Em seguida, verificar o número de riscos necessários.

Se número de riscos é menor que a ordem da matriz, ela deverá ser novamente reduzida. No exemplo, foram necessários 2 riscos para cobrir todos os zeros, mas a ordem da matriz é 3.

CORRETOR	IMÓVEL		
	1	2	3
1	0	0	2
2	1	5	0
3	0	3	0

-2

+2

Neste caso soma pois a célula está na interseção dos riscos

4º Passo – Identificar o **menor valor** que não esteja sendo atravessado por algum risco. No exemplo é o valor 2, que subtrai os valores não cobertos pelo risco.

Nas **células** que estão **na interseção de riscos de linhas e de colunas** o valor mínimo deve ser **somado**. No exemplo, a célula dada pelo Corretor 1 x Imóvel 3 está na interseção dos riscos, por isso **o valor mínimo (2) deve ser somado** ao valor da célula (antes era 0).



6.2. Algoritmo Húngaro

Exemplo 2 – Atribuição de Corretores

CORRETOR	IMÓVEL		
	1	2	3
1	0	0	2
2	1	5	0
3	0	3	0

5º Passo – Retorna-se ao passo de verificação da cobertura mínima. Como o número mínimo de riscos para cobrir todos os zeros da matriz é três (igual a sua ordem), o procedimento encerra.

CORRETOR	IMÓVEL		
	1	2	3
1	0	0	2
2	1	5	0
3	0	3	0

Considerando os zeros não conflitantes para cada corretor, a combinação (Corretor, Imóvel) que corresponde ao menor custo total de distância é:

- Corretor 1, Imóvel 2 = 15
- Corretor 2, Imóvel 3 = 5
- Corretor 3, Imóvel 1 = 6

Total (km) = 26

CORRETOR	IMÓVEL		
	1	2	3
1	10	15	9
2	9	18	5
3	6	14	3



6.2. Algoritmo Húngaro

Exemplo 2 – Atribuição de Corretores

CORRETOR	IMÓVEL		
	1	2	3
1	0	0	2
2	1	5	0
3	0	3	0

5º Passo – Retorna-se ao passo de verificação da cobertura mínima. Como o número mínimo de riscos para cobrir todos os zeros da matriz é três (igual a sua ordem), o procedimento encerra.

CORRETOR	IMÓVEL		
	1	2	3
1	0	0	2
2	1	5	0
3	0	3	0

CORRETOR	IMÓVEL		
	1	2	3
1	10	15	9
2	9	18	5
3	6	14	3

REFERÊNCIAS

UNIVESP – Método Húngaro

<https://www.youtube.com/watch?v=EmXJh7g5CwQ>

Hungarian Algorithm and Its Applications in Computer Vision

<https://towardsdatascience.com/hungarian-algorithm-and-its-applications-in-computer-vision/>

Solução pelo Método Húngaro

<https://www.metodosnumericos.com.br/metodohungaro.html>



6.2. Algoritmo Húngaro

Exemplo 3 – Venda de Moedas

Um negociante de moedas vai vender quatro moedas raras em um leilão eletrônico. Ele recebe propostas para cada uma das quatro moedas de cinco interessados, mas estes interessados também afirmam que podem honrar no máximo uma das propostas. A tabela abaixo descreve as propostas.

		MOEDA			
		1	2	3	4
COLECIONADOR	1	150	65	210	135
	2	175	75	230	155
	3	135	85	200	140
	4	140	70	190	130
	5	170	50	200	160



6.2. Algoritmo Húngaro

Exemplo 3 – Venda de Moedas

Um negociante de moedas vai vender quatro moedas raras em um leilão eletrônico. Ele recebe propostas para cada uma das quatro moedas de cinco interessados, mas estes interessados também afirmam que podem honrar no máximo uma das propostas. A tabela abaixo descreve as propostas.

		MOEDA			
		1	2	3	4
COLECIONADOR	1	150	65	210	135
	2	175	75	230	155
	3	135	85	200	140
	4	140	70	190	130
	5	170	50	200	160

AJUSTES NA MATRIZ DE CUSTOS

A matriz de custo não é quadrada.

O Algoritmo Húngaro clássico resolve o problema de atribuição em uma matriz $n \times n$.

Se a matriz de custos não for quadrada o problema fica desbalanceado (ex. mais colecionadores que moedas).

Para aplicar o algoritmo sem alterar sua lógica, adiciona-se uma linha ou coluna fictícia. Para o caso, adicionar uma coluna representando uma moeda fictícia.



6.2. Algoritmo Húngaro

Exemplo 3 – Venda de Moedas

Um negociante de moedas vai vender quatro moedas raras em um leilão eletrônico. Ele recebe propostas para cada uma das quatro moedas de cinco interessados, mas estes interessados também afirmam que podem honrar no máximo uma das propostas. A tabela abaixo descreve as propostas.

		MOEDA			
		1	2	3	4
COLECIONADOR	1	150	65	210	135
	2	175	75	230	155
	3	135	85	200	140
	4	140	70	190	130
	5	170	50	200	160

AJUSTES NA MATRIZ DE CUSTOS

O problema é de maximização.

O objetivo do proprietário das moedas é maximizar seu lucro com as vendas.

Deve-se multiplicar todos os elementos da matriz de custo por -1 para converter o problema de maximização para minimização, que é a forma adotada pelo algoritmo Húngaro clássico. Após a conversão a técnica continua a mesma.



6.2. Algoritmo Húngaro

Exemplo 3 – Venda de Moedas

		MOEDA				
		1	2	3	4	5
COLECCIONADOR	1	-150	-65	-210	-135	0
	2	-175	-75	-230	-155	0
	3	-135	-85	-200	-140	0
	4	-140	-70	-190	-130	0
	5	-170	-50	-200	-160	0

AJUSTES NA MATRIZ DE CUSTOS

Inclusão da **coluna 5** representa uma moeda fictícia. A atribuição da moeda 5 a um colecionador, significa que ele fez o pior lance, logo nenhuma moeda será vendida para ele.

Multiplicação de todos os elementos da matriz de custo por -1 .



6.2. Algoritmo Húngaro

Exemplo 3 – Venda de Moedas

COLECIONADOR	MOEDA				
	1	2	3	4	5
1	-150	-65	-210	-135	0
2	-175	-75	-230	-155	0
3	-135	-85	-200	-140	0
4	-140	-70	-190	-130	0
5	-170	-50	-200	-160	0

AJUSTES NA MATRIZ DE CUSTOS

Inclusão da coluna 5 representa uma moeda fictícia. A atribuição da moeda 5 a um colecionador, significa que ele fez o pior lance, logo nenhuma moeda será vendida para ele.

Multiplicação de todos os elementos da matriz de custo por -1 .

COLECIONADOR	MOEDA				
	1	2	3	4	5
1	60	145	0	75	210
2	55	155	0	85	230
3	65	115	0	60	200
4	50	120	0	60	190
5	30	150	0	40	200

1º Passo – Subtrair o menor valor de cada linha de todos os valores da mesma linha.

Exemplos:

- 1ª Linha: $C1-M1 = -150 - (-210) = 60$
- 2ª Linha: $C2-M1 = -175 - (-230) = 55$
- 3ª Linha: $C3-M1 = -135 - (-200) = 65$



6.2. Algoritmo Húngaro

Exemplo 3 – Venda de Moedas

COLECIONADOR	MOEDA				
	1	2	3	4	5
1	60	145	0	75	210
2	55	155	0	85	230
3	65	115	0	60	200
4	50	120	0	60	190
5	30	150	0	40	200

2º Passo – Subtrair o menor valor de cada coluna de todos os valores da mesma coluna.

Exemplo:

- 1ª Coluna: $C1-M1 = 60 - 30 = 30$
- 2ª Coluna: $C1-M2 = 145 - 115 = 30$

COLECIONADOR	MOEDA				
	1	2	3	4	5
1	30	30	0	35	20
2	25	40	0	45	40
3	35	0	0	20	10
4	20	5	0	20	0
5	0	35	0	0	10



6.2. Algoritmo Húngaro

Exemplo 3 – Venda de Moedas

COLECCIONADOR	MOEDA				
	1	2	3	4	5
1	60	145	0	75	210
2	55	155	0	85	230
3	65	115	0	60	200
4	50	120	0	60	190
5	30	150	0	40	200

COLECCIONADOR	MOEDA				
	1	2	3	4	5
1	30	30	0	35	20
2	25	40	0	45	40
3	35	0	0	20	10
4	20	5	0	20	0
5	0	35	0	0	10

2º Passo – Subtrair o menor valor de cada coluna de todos os valores da mesma coluna.

Exemplo:

- 1ª Coluna: $C1-M1 = 60 - 30 = 30$
- 2ª Coluna: $C1-M2 = 145 - 115 = 30$

3º Passo – Riscar as linhas ou colunas de modo a cobrir todos os valores zero com a menor quantidade de riscos.



6.2. Algoritmo Húngaro

Exemplo 3 – Venda de Moedas

COLECIONADOR	MOEDA				
	1	2	3	4	5
1	60	145	0	75	210
2	55	155	0	85	230
3	65	115	0	60	200
4	50	120	0	60	190
5	30	150	0	40	200

COLECIONADOR	MOEDA				
	1	2	3	4	5
1	30	30	0	35	20
2	25	40	0	45	40
3	35	0	0	20	10
4	20	5	0	20	0
5	0	35	0	0	10

2º Passo – Subtrair o menor valor de cada coluna de todos os valores da mesma coluna.

Exemplo:

- 1ª Coluna: $C1-M1 = 60 - 30 = 30$
- 2ª Coluna: $C1-M2 = 145 - 115 = 30$

3º Passo – Riscar as linhas ou colunas de modo a cobrir todos os valores zero com a menor quantidade de riscos.

4º Passo – Teste de Solução ótima: como o número mínimo de riscos em linhas e colunas para cobrir todos os zeros é 4 (linhas 3, 4 e 5 e coluna 3) é inferior a 5 (ordem da matriz), então ainda não é possível uma alocação ótima de zeros.



6.2. Algoritmo Húngaro

Exemplo 3 – Venda de Moedas

COLECIONADOR	MOEDA				
	1	2	3	4	5
1	30	30	0	35	20
2	25	40	0	45	40
3	35	0	0	20	10
4	20	5	0	20	0
5	0	35	0	0	10

5º Passo – Identificar o **menor valor** não riscado (**20**) e **subtrair de todos** os valores **não riscados**. Nos valores **riscados por linhas e colunas** deve-se **somar** com o **menor valor**. Em seguida retorna ao 3º Passo.

COLECIONADOR	MOEDA				
	1	2	3	4	5
1	10	10	0	15	0
2	5	20	0	25	20
3	35	0	20	20	10
4	20	5	20	20	0
5	0	35	20	0	10

-20

+20



6.2. Algoritmo Húngaro

Exemplo 3 – Venda de Moedas

COLECCIONADOR	MOEDA				
	1	2	3	4	5
1	30	30	0	35	20
2	25	40	0	45	40
3	35	0	0	20	10
4	20	5	0	20	0
5	0	35	0	0	10

5º Passo – Identificar o menor valor não riscado (**20**) e subtrair de todos os valores não riscados. Nos valores riscados por linhas e colunas deve-se somar com o menor valor. Em seguida retorna ao 3º Passo.

COLECCIONADOR	MOEDA				
	1	2	3	4	5
1	10	10	0	15	0
2	5	20	0	25	20
3	35	0	20	20	10
4	20	5	20	20	0
5	0	35	20	0	10

3º e 4º Passos – novamente, são riscadas as linhas ou colunas que cobrem todos os valores zero com a menor quantidade de riscos. Como o número de riscos é menor que a ordem da matriz ($4 < 5$) realiza-se novamente o ajuste no 5º Passo.



6.2. Algoritmo Húngaro

Exemplo 3 – Venda de Moedas

COLECIONADOR	MOEDA				
	1	2	3	4	5
1	10	10	0	15	0
2	5	20	0	25	20
3	35	0	20	20	10
4	20	5	20	20	0
5	0	35	20	0	10

5º Passo – Identificar o **menor valor** não riscado (**5**) e **subtrair de todos** os valores **não riscados**. Nos valores **riscados por linhas e colunas** deve-se **somar** com o **menor valor**. Em seguida retorna ao 3º Passo.

COLECIONADOR	MOEDA				
	1	2	3	4	5
1	5	5	0	10	0
2	0	15	0	20	20
3	35	0	25	20	15
4	15	0	20	15	0
5	0	35	25	0	15

-5

+5



6.2. Algoritmo Húngaro

Exemplo 3 – Venda de Moedas

COLECCIONADOR	MOEDA				
	1	2	3	4	5
1	10	10	0	15	0
2	5	20	0	25	20
3	35	0	20	20	10
4	20	5	20	20	0
5	0	35	20	0	10

5º Passo – Identificar o menor valor não riscado (5) e subtrair de todos os valores não riscados. Nos valores riscados por linhas e colunas deve-se somar com o menor valor. Em seguida retorna ao 3º Passo.

COLECCIONADOR	MOEDA				
	1	2	3	4	5
1	5	5	0	10	0
2	0	15	0	20	20
3	35	0	25	20	15
4	15	0	20	15	0
5	0	35	25	0	15

3º e 4º Passos – novamente, são riscadas as linhas ou colunas que cobrem todos os valores zero com a menor quantidade de riscos. Como o número de riscos é igual a ordem da matriz (**5 = 5**), então é possível a alocação ótima e o algoritmo encerra.



6.2. Algoritmo Húngaro

Exemplo 3 – Venda de Moedas

		MOEDA				
		1	2	3	4	5
COLECIONADOR	1	5	5	0	10	0
	2	0	15	0	20	20
	3	35	0	25	20	15
	4	15	0	20	15	0
	5	0	35	25	0	15

		MOEDA			
		1	2	3	4
COLECIONADOR	1	150	65	210	135
	2	175	75	230	155
	3	135	85	200	140
	4	140	70	190	130
	5	170	50	200	160

A alocação ótima de zeros indica que a maior quantia que o negociante poderia conseguir é dada pelas seguintes vendas:

- Colecionador 1, Moeda 3 = 210
- Colecionador 2, Moeda 1 = 175
- Colecionador 3, Moeda 2 = 85
- Colecionador 5, Moeda 4 = 160

Total (R\$) = **630**

Obs. Ao Colecionador 4 não seria vendida nenhuma moeda.

Perguntas? Sugestões?

